NonlinearOscillatorLab: A Python and Wolfram Language Toolkit for Energy-Based Classification, Accurate Period Computation and Closed-Form Solution of Duffing and Cubic–Quintic Oscillators

Alvaro H. Salas
Department of Mathematics and Statistics
Universidad Nacional de Colombia, Sede Manizales, Colombia
Corresponding author: ahsalass@unal.edu.co

Original Software Publication, SoftwareX — October 2026

Abstract

NonlinearOscillatorLab is an MIT-licensed toolkit for the autonomous conservative oscillator $\ddot{x}+ax+bx^3+cx^5=0$, distributed as two cross-validated implementations: a Python package and a Wolfram Language package for Mathematica. Both determine equilibria and turning points, detect separatrix energies, compute periods on bounded energy components and integrate initial-value problems. Periods are obtained from a factored, singularity-free integrand $1/\sqrt{2Q(x)}$, where $E-V(x)=(x-x_-)(x_+-x)Q(x)$, which gives median relative errors of order 10^{-15} and at most 3×10^{-11} up to 10^{-6} relative distance from a separatrix, and arbitrary precision in the Wolfram implementation. The Wolfram package also provides the author's closed-form representation $x=Ay/\sqrt{1+\lambda y^2+\mu y^4}$, where y is a Jacobi-elliptic solution of a cubic Duffing equation. Verification combines closed-form elliptic periods, the logarithmic separatrix law $T\simeq k\ln(16/|E-E_s|)$, 32-digit reference trajectories and a seeded randomised campaign. All figures in this paper are generated by the bundled Mathematica scripts.

Keywords: research software; Duffing oscillator; cubic–quintic oscillator; Jacobi elliptic functions; energy methods; period computation; Wolfram Language; reproducibility.

Code metadata

Current code version	v0.1.0
Permanent link to code repository	https://github.com/alvarohumberto/NonlinearOscillatorLab
Reproducible capsule	Not available
Legal code license	MIT
Code versioning system	git
Languages, tools, services	Python (NumPy, SciPy, pytest); Wolfram Language (Mathematica / Wolfram Engine)
Requirements	Python 3.10+; Mathematica or Wolfram Engine 14.0+ (tested with 15.0 on Windows 64-bit)
Developer documentation	README.md; demo notebook in `mathematica/`
Support email	ahsalass@unal.edu.co

1 Motivation and significance

Since Duffing's monograph [1], one-dimensional nonlinear oscillators have remained standard reference systems in mechanics, optics, plasma physics and computational dynamics. Despite the apparent simplicity of the Duffing equation, multiple wells, turning points and separatrices make even the period a quantity that depends on the connected component of the energy surface containing the initial state. Cubic–quintic restoring forces add triple wells and further orbit families. A dependable tool must therefore keep numerical time integration, root identification, energy diagnostics, period evaluation and analytic solution apart, rather than assume that every initial-value problem is periodic or that one formula covers every orbit.

NonlinearOscillatorLab provides these tasks through a small, inspectable API in two languages. The Python package, built on NumPy and SciPy [6, 5], serves scripted and continuous-integration workflows; the Wolfram Language package adds exact and arbitrary-precision arithmetic, Jacobi-elliptic special functions and the author's closed-form solver for the cubic–quintic equation. Because both implementations expose the same operations, every published number can be computed twice by independent code. This cross-verification revealed a loss of accuracy of the endpoint-substitution quadrature near separatrices (up to 1.7×10^{-5} relative error in a pre-release version), which motivated the factored integrand now used by both implementations (Section 2.2).

2 Software description

2.1 Model and architecture

The package treats the initial-value problem

$$\ddot{x} + ax + bx^3 + cx^5 = 0, \qquad x(0) = x_0, \quad \dot{x}(0) = v_0, \tag{1}$$

with potential and conserved energy

$$V(x) = \frac{a}{2}x^2 + \frac{b}{4}x^4 + \frac{c}{6}x^6, \qquad E = \frac{1}{2}\dot{x}^2 + V(x). \tag{2}$$

Table 1 lists the corresponding functions; Appendix A gives the Wolfram Language reference, usage examples and complete source code. Equilibria solve $x(a + bx^2 + cx^4) = 0$ and turning points solve $V(x) = E$, both through polynomial roots in $y = x^2$; inexact roots are polished by Newton steps. In the Wolfram implementation exact coefficients yield exact equilibria (e.g. $\{-1, -1/\sqrt{3}, 0, 1/\sqrt{3}, 1\}$ for the triple well $a = 1, b = -4, c = 3$). `classify_energy` reports minima, maxima and whether E coincides with a barrier energy; a period is returned only when a unique bounded component $[x_-, x_+]$ containing x_0 is identified, and a failure is reported otherwise.

Table 1: Corresponding functions of the two implementations.

Task	Python (`nonlinear_oscillator_lab`)	Wolfram Language
Model, V, force, E	`Model(a,b,c)`	`NOLModel[a,b,c]`, `NOLPotential`, `NOLEnergy`
Equilibria	`stationary_points`	`NOLStationaryPoints` (exact if possible)
Energy classification	`classify_energy`	`NOLClassifyEnergy`
Turning points	`turning_points`	`NOLTurningPoints`, `NOLBoundedComponent`
Period	`period`	`NOLPeriod` (any `WorkingPrecision`)
Closed-form Duffing period	—	`NOLDuffingExactPeriod`
Closed-form solution	—	`NOLExactSolution`, `NOLDuffingSolution`
Time integration	`simulate` (DOP853)	`NOLSimulate` (explicit RK, order 8)
Energy drift	`energy_drift`	`NOLEnergyDrift`
Phase portrait	—	`NOLPhasePortrait`

2.2 Singularity-free period quadrature

For a regular orbit oscillating in $[x_-, x_+]$ the period is $T(E) = 2\int_{x_-}^{x_+} dx/\sqrt{2(E - V(x))}$. Since $x_\pm$ are simple roots of the polynomial $E - V$, one may write exactly

$$E - V(x) = (x - x_-)(x_+ - x)\, Q(x), \qquad Q > 0 \text{ on } [x_-, x_+], \tag{3}$$

where Q is a polynomial of degree at most four obtained by polynomial division. With $x = \bar{x} + h\sin\theta$, $\bar{x} = (x_+ + x_-)/2$, $h = (x_+ - x_-)/2$, one has $(x - x_-)(x_+ - x) = h^2\cos^2\theta$ and therefore

$$T(E) = 2\int_{-\pi/2}^{\pi/2} \frac{d\theta}{\sqrt{2\, Q(\bar{x} + h\sin\theta)}}. \tag{4}$$

The integrand of (4) is smooth and free of the cancellation $E - V(x) \to 0$ that affects the plain substitution at the endpoints. Near a separatrix Q develops a narrow minimum where the orbit passes the barrier; the quadrature is split at the corresponding angles $\theta_k = \arcsin((x_k - \bar{x})/h)$ of the stationary points $x_k \in (x_-, x_+)$. In the Wolfram implementation all steps run at a user-selected `WorkingPrecision` with 20 guard digits.

2.3 Closed-form references and solutions

For $c = 0$ the package provides the classical elliptic periods

$$T = \frac{4K(m)}{\sqrt{a + bA^2}}, \qquad m = \frac{bA^2}{2(a + bA^2)} \qquad (x = A\,\mathrm{cn}(\omega t \,|\, m)), \tag{5}$$

$$T = \frac{2K(m)}{A\sqrt{b/2}}, \qquad m = 2 + \frac{2a}{bA^2} \qquad (x = A\,\mathrm{dn}(\omega t \,|\, m),\ \text{intra-well}), \tag{6}$$

where A is the right turning point and K the complete elliptic integral of the first kind in the parameter convention [3, 4]. Near the barrier of the double well $a = -1, b = 1$ the period obeys $T \simeq k\ln(16/|E - E_s|)$, with $k = 1$ for intra-well and $k = 2$ for inter-well orbits; both laws are used as independent tests.

For the full cubic–quintic equation, `NOLExactSolution` implements the author's representation

$$x(t) = \frac{A\, y(t)}{\sqrt{1 + \lambda y(t)^2 + \mu y(t)^4}}, \qquad \ddot{y} + p\, y + q\, y^3 = 0, \tag{7}$$

in which the parameters $(A, \lambda, \mu, p, q, \dot{y}(0))$ follow from the real roots of the cubic $hP^3 - aP^2 - \frac{b}{2}P - \frac{c}{3} = 0$, $h = 2E$ [2]. Two families arise ($\mu = 0$ and $\mu \neq 0$). The Duffing solution y is selected from the cn, dn, sech and sn/cn branches by `NOLDuffingSolution`, and a time shift handles initial states at rest at a pole of y. Candidates are accepted only after consistency checks on $x(0)$, the local Taylor expansion and the first integral $\dot{x}^2 = 2(E - V(x))$; when no real representation passes them a `Failure` object is returned.

2.4 Simulation and diagnostics

Trajectories are integrated with SciPy's `DOP853` [5, 7] or with Mathematica's [8] eighth-order explicit Runge–Kutta method with dense output of the same order (`InterpolationOrder->All`). The normalised energy drift is $\delta_E = \max_j |E_j - E_0|/(1 + |E_0|)$.

3 Illustrative examples

Figure 1 shows the five reference potentials used throughout, with their equilibria computed by `NOLStationaryPoints`; Figure 2 shows phase portraits with separatrices obtained from `NOLClassifyEnergy`. Figure 3 compares the closed form (7) with numerical solutions for the demonstration case $a = 1, b = 0.4, c = 0.2$, $x_0 = 0.8$, $v_0 = 0.1$ (representation: family II, `cn` branch) and for triple- and double-well orbits; panels (b) and (d) give pointwise errors against a 32-digit `NDSolve` reference. Figure 4 shows period–energy curves of all eight orbit families, including the divergence at the separatrices.

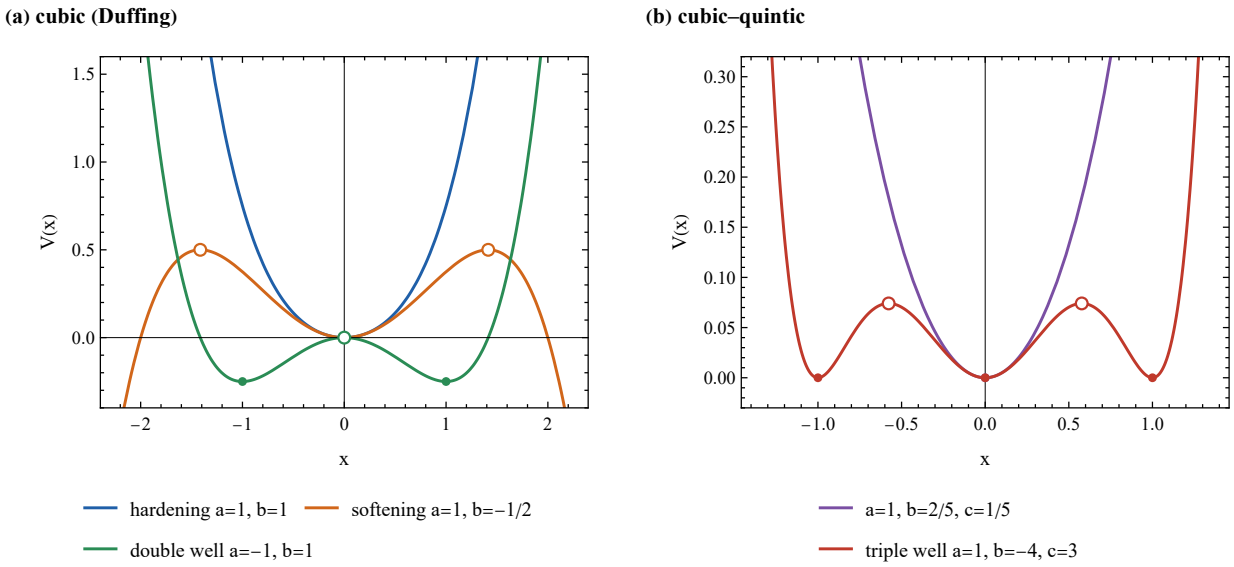


Figure 1: Potentials $V(x)$ for (a) the Duffing cases and (b) the cubic–quintic cases. Filled dots: minima; open circles: maxima (barriers).

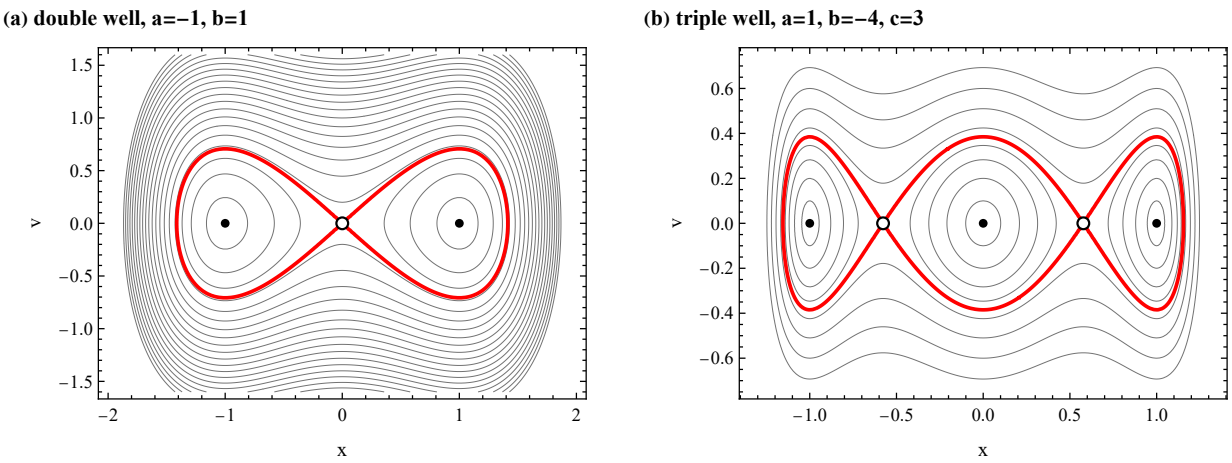


Figure 2: Energy level sets in the phase plane; separatrices (red) pass through the barriers. (a) Double well; (b) triple well with barrier energy 2/27.

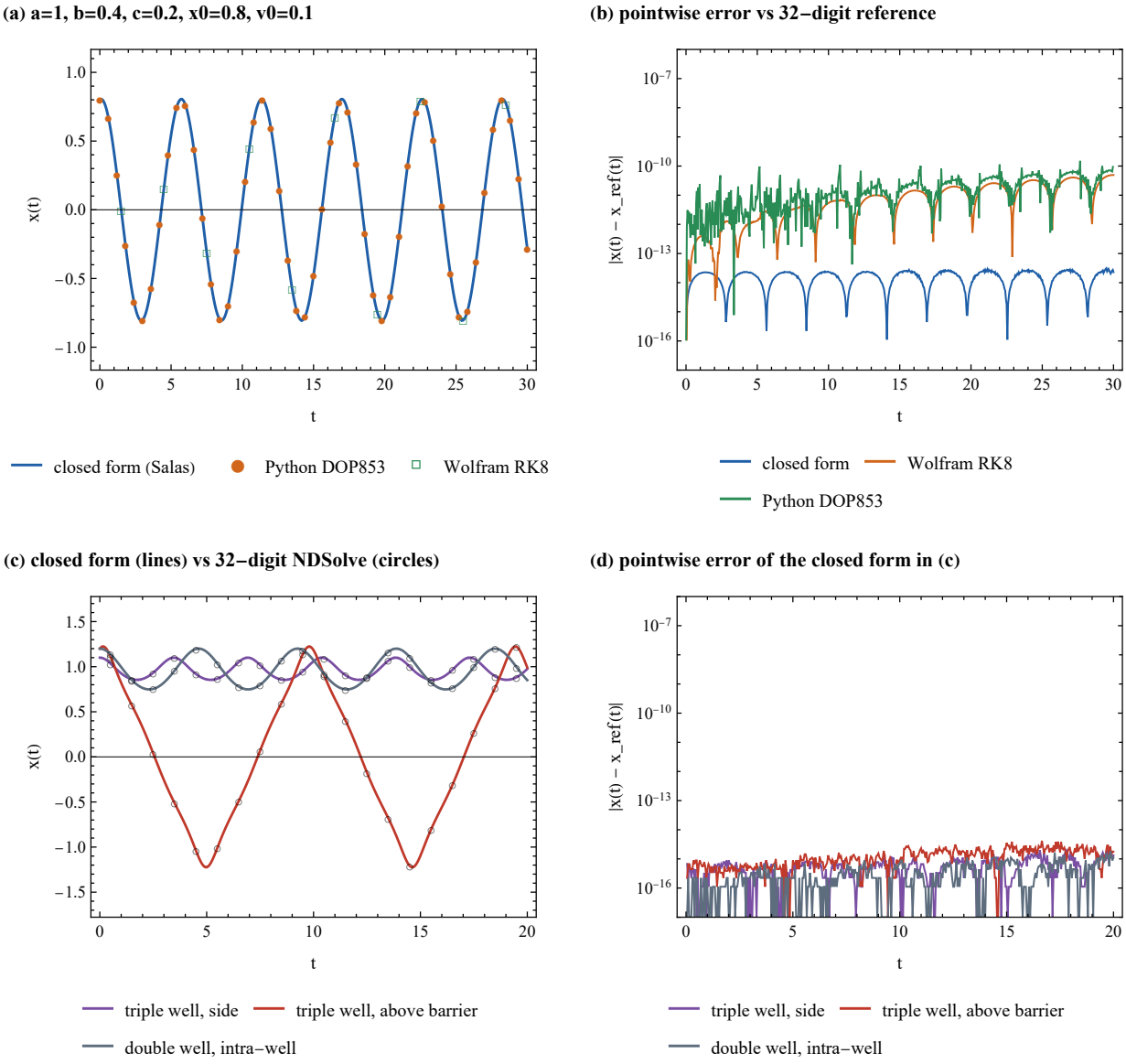


Figure 3: (a) Closed-form solution (7) and numerical solutions for $a = 1, b = 0.4, c = 0.2, x_0 = 0.8, v_0 = 0.1$. (b) Pointwise deviation from a 32-digit reference for the closed form, the Wolfram RK8 integrator and the Python DOP853 integrator. (c) Closed form versus 32-digit reference for three further orbits and (d) its pointwise error.

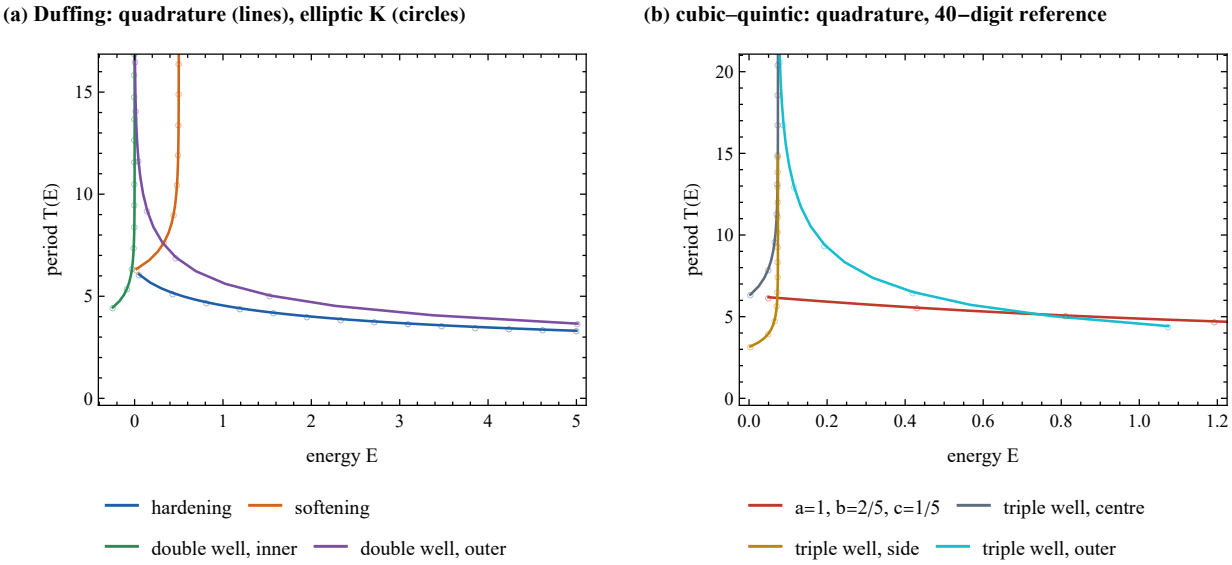


Figure 4: Period–energy curves. Lines: machine-precision quadrature (4); circles: (a) closed-form elliptic periods, (b) 40-digit quadrature.

4 Verification

Automated tests. The Python suite contains 15 tests, and the Wolfram suite (`RunTests.wls`) contains 31 tests; on 15.0.0 for Microsoft Windows 31 of 31 pass, and all Python tests pass. They cover the harmonic reference, exact equilibria, separatrix detection, elliptic periods (machine and 50 digits), the elliptic solutions, the separatrix law, orbit closure $x(T) = x_0$, the closed form (7) on eight orbits, and agreement with the Python periods.

Period accuracy. Figure 5(a) shows the relative error of 320 periods (8 families, 40 energies each, approaching the separatrices up to 10^{-6} relative distance) against closed-form elliptic values ($c = 0$) or 40-digit quadrature ($c \neq 0$). The factored integrand gives median errors of 6.6×10^{-16} (Python) and 1.5×10^{-15} (Wolfram), and maximum errors of 1.2×10^{-11} and 2.4×10^{-11}, compared with 1.7×10^{-5} for the endpoint-substitution quadrature. Figure 5(b) confirms the separatrix law down to $|E - E_s| = 10^{-16}$. Table 2 lists nine benchmark periods to 16 digits.

Closed-form solver. Figure 6 reports a seeded campaign (`SeedRandom[20261019]`) of 200 random problems with $a, b \in [-12, 12]$, $c \in [0, 1]$, $x_0 \in [-2, 0]$, $v_0 \in [-2, 2]$, the parameter ranges of the author's original test script. A real representation was found in 200 cases (median construction time 174 ms); on $[0, 20]$ the deviation from a 28-digit reference has median 3.9×10^{-14}, 95th percentile 2.8×10^{-12} and maximum 3.6×10^{-9}, and 100% of all cases are below 10^{-8}.

Precision and tolerance. Figure 7(a) shows that the number of correct digits of the Wolfram period, measured against an 80-digit value, is at least the requested `WorkingPrecision`, and Figure 7(b) shows the RK8 energy drift over $[0, 200]$ as a function of `PrecisionGoal`.

Table 2: Benchmark periods (T, 40-digit Wolfram quadrature, 16 digits shown), relative deviation of the Python and Wolfram machine-precision periods, and maximum deviation of the closed form (7) on $[0, 20]$ from a 32-digit reference.

case	a	b	c	x_0	v_0	T	Python	Wolfram	closed form
harmonic	4	0	0	$\frac{2}{5}$	0	3.141592653589793	0	1.1×10^{-15}	1.2×10^{-15}
hardening	1	1	0	1	0	4.768022029102461	0	1.1×10^{-15}	5.9×10^{-15}
softening	1	$-\frac{1}{2}$	0	$\frac{9}{10}$	0	7.560247907820581	0	1.1×10^{-15}	3.6×10^{-15}
dw_inner	-1	1	0	$\frac{6}{5}$	0	4.623905221811913	3.8×10^{-16}	7.7×10^{-16}	1.8×10^{-15}
dw_outer	-1	1	0	$\frac{8}{5}$	0	7.386828150976248	2.4×10^{-16}	9.6×10^{-16}	5.1×10^{-15}
cq_demo	1	$\frac{2}{5}$	$\frac{1}{5}$	$\frac{4}{5}$	$\frac{1}{10}$	5.633261676532774	1.6×10^{-16}	9.5×10^{-16}	2.8×10^{-14}
tw_center	1	-4	3	$\frac{3}{10}$	0	7.295155971525862	1.2×10^{-16}	1.2×10^{-15}	1.2×10^{-15}
tw_side	1	-4	3	$\frac{11}{10}$	0	3.460792103704557	1.3×10^{-16}	1.8×10^{-15}	2.2×10^{-15}
tw_outer	1	-4	3	$\frac{6}{5}$	$\frac{3}{10}$	9.646253874918385	2.0×10^{-15}	2.0×10^{-15}	4.4×10^{-15}

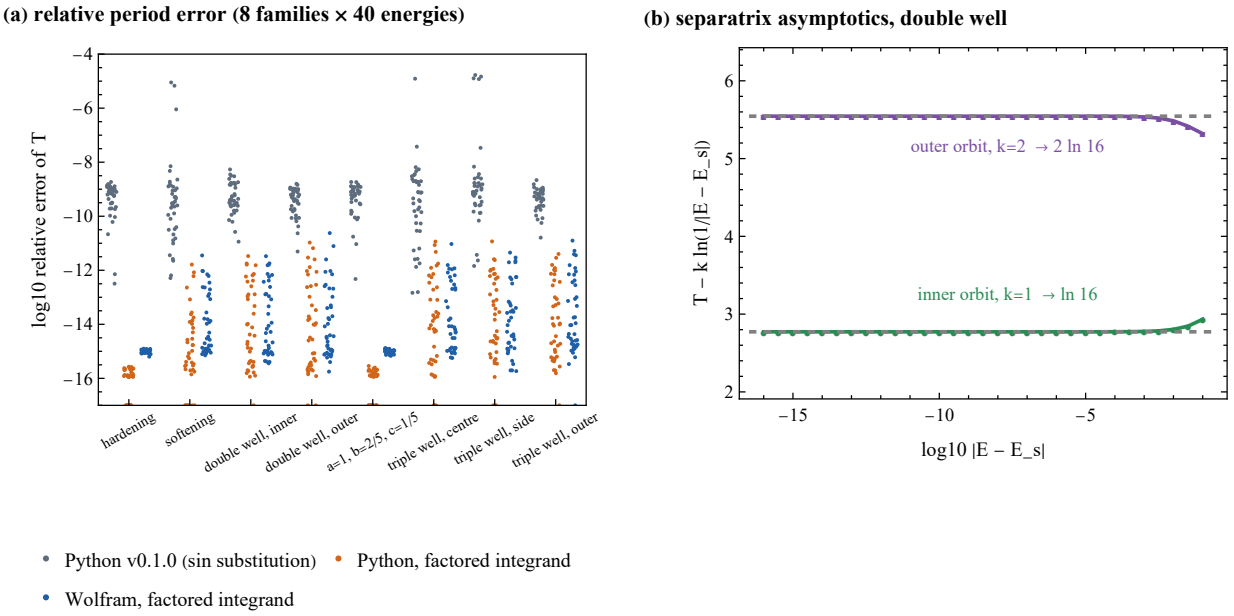


Figure 5: (a) Relative period error for 8 families × 40 energies: endpoint-substitution quadrature (pre-release Python), factored quadrature in Python and in the Wolfram Language. (b) $T - k \ln(1/|E - E_s|)$ for the double well at 40 digits, converging to $\ln 16$ ($k = 1$) and $2 \ln 16$ ($k = 2$).

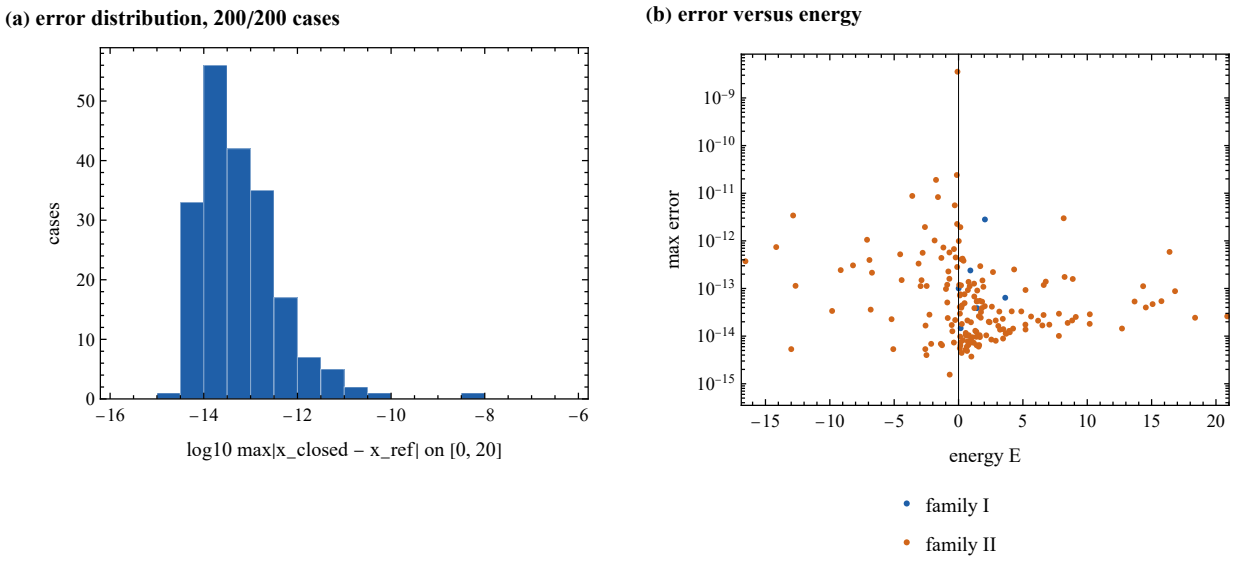


Figure 6: Randomised test of the closed-form solver (7): (a) distribution of the maximum deviation from a 28-digit reference on $[0, 20]$; (b) deviation versus energy, by representation family.

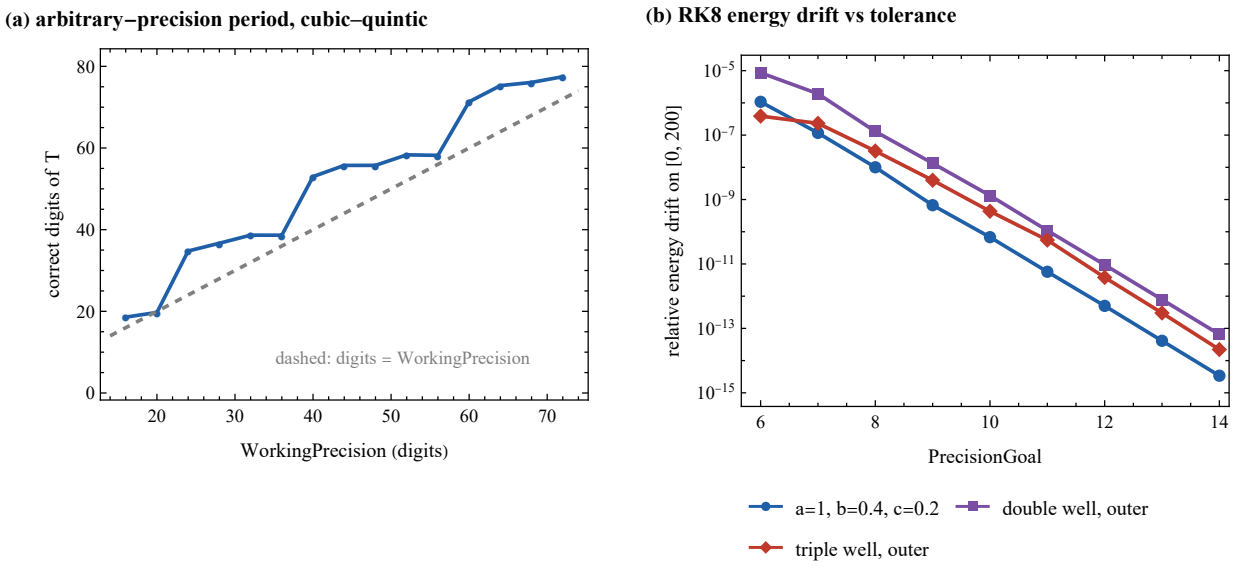


Figure 7: (a) Correct digits of the arbitrary-precision period versus `WorkingPrecision` for $a = 1, b = 2/5, c = 1/5, x_0 = 4/5, v_0 = 1/10$. (b) Energy drift of the RK8 integrator versus `PrecisionGoal`.

5 Impact and reuse potential

Researchers can use the package as an independent control when developing perturbation series, harmonic-balance, Jacobi-elliptic or other analytic approximations for Duffing-type equations, since it supplies closed forms, machine-precision periods and arbitrary-precision references within one API. The dual implementation lets a computation be repeated in a second language, and the scripts that produce every figure and table of this paper are part of the distribution (`mathematica/RunAll.wls`, `benchmarks/`). Teaching applications include phase-plane interpretation, separatrix asymptotics,

multi-well potentials and numerical-tolerance studies. No claims of existing adoption or external users are made for this initial release.

6 Limitations and future work

The software does not handle damping, forcing, unilateral contact or coupled oscillators, and it provides no interval-certified error bounds. Period evaluation requires simple turning points; exactly at a separatrix the period is reported as infinite and multiple-root cases are rejected. The closed-form solver is currently available only in the Wolfram Language; its coverage is empirical (Figure 6), and for initial states without a real representation it returns a failure rather than an approximation. Planned work includes a Python port of the closed-form solver, event handling and interval enclosures.

Availability and reproducibility

The Python package, the Wolfram Language package, test suites, benchmark scripts and the figure-generating scripts accompany this manuscript as an MIT-licensed archive. Running `wolframscript -file mathematica/RunAll.wls` regenerates every figure and table. Appendix A documents the Wolfram Language package, its usage, and lists its complete source code. The source code is publicly available at https://github.com/alvarohumberto/NonlinearOscillatorLab.

CRediT authorship contribution statement

Alvaro H. Salas: Conceptualization, Methodology, Software, Validation, Writing – original draft, Writing – review and editing.

Declaration of competing interests

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work.

Funding

No external funding is declared.

Data availability

No external research datasets were used. All data shown are generated by the scripts included with the code archive.

References

[1] G. Duffing, *Erzwungene Schwingungen bei veränderlicher Eigenfrequenz und ihre technische Bedeutung*, Vieweg, Braunschweig, 1918.

[2] A. H. Salas, **[ADD: reference for the closed-form cubic–quintic representation** $x = Ay/\sqrt{1+\lambda y^2+\mu y^4}$**]**.

[3] P. F. Byrd, M. D. Friedman, *Handbook of Elliptic Integrals for Engineers and Scientists*, 2nd ed., Springer, Berlin, 1971.

[4] NIST Digital Library of Mathematical Functions, Chapter 22: Jacobian Elliptic Functions, https://dlmf.nist.gov/22.

[5] P. Virtanen et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, *Nature Methods* 17 (2020) 261–272. https://doi.org/10.1038/s41592-019-0686-2.

[6] C. R. Harris et al., Array programming with NumPy, *Nature* 585 (2020) 357–362. https://doi.org/10.1038/s41586-020-2649-2.

[7] E. Hairer, S. P. Nørsett, G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer, Berlin, 1993.

[8] Wolfram Research, Inc., Mathematica, Version 15.0, Champaign, IL, 2026.

A Wolfram Language (Mathematica) implementation and usage

This appendix documents the Wolfram Language package `NonlinearOscillatorLab.wl` and the scripts that produce every figure and table of the paper. The code was run with Mathematica 15.0 on Windows 64-bit, and Mathematica or Wolfram Engine 14.0 or later is required. The complete source is listed in Sections A.4–A.6; the same files are in the `mathematica/` folder of the software archive.

A.1 Installation and running

No installation is needed. Download or clone the repository (https://github.com/alvarohumberto/NonlinearOscillatorLab), or copy the folder `mathematica/` anywhere, and load the package by file name:

```
Get["C:/path/to/NonlinearOscillatorLab/mathematica/NonlinearOscillatorLab.wl"]
NOLVersion        (* "0.1.0" *)
```

The verification suite, the figures and the tables are reproduced from a terminal with `wolframscript`,

```
wolframscript -file mathematica/RunTests.wls       (* 31 tests -> test_report.txt / .csv *)
wolframscript -file mathematica/MakeFigures.wls    (* figures/*.pdf, *.png; tables/*.csv, *.tex *)
wolframscript -file mathematica/RunAll.wls         (* both *)
```

or from a notebook with `Get[".../mathematica/RunAll.wls"]`. `MakeFigures.wls` reads the Python reference data in `benchmarks/` and `examples/orbit.csv`, so Python is not needed to rebuild the figures. A full run takes about 3–4 minutes on a laptop; most of the time is spent in the 200 reference integrations of the randomised campaign. `MakeDemoNotebook.wls` writes the tutorial notebook `NonlinearOscillatorLab_Demo.nb`.

A.2 Function reference

Table 3 summarises the public functions. A model is the inert expression `NOLModel[a,b,c]` (`NOLModel[a,b]` sets $c = 0$). Exact coefficients and initial data (integers, rationals, radicals) give exact equilibria and turning points and allow any `WorkingPrecision`; machine numbers give machine-precision results. Functions that cannot return a meaningful value return a `Failure` object and issue a message, so results can be tested with `FailureQ`.

Table 3: Public functions of `NonlinearOscillatorLab.wl`.

Call	Result
`NOLPotential[m,x]`, `NOLForce[m,x]`, `NOLEnergy[m,x,v]`, `NOLCurvature[m,x]`	$V(x)$, $-V'(x)$, $\tfrac{1}{2}v^2+V(x)$, $V''(x)$
`NOLStationaryPoints[m]`	sorted real roots of $V'(x)=0$
`NOLClassifyEnergy[m,x0,v0]`	Association with keys `"Energy"`, `"Equilibria"`, `"Minima"`, `"Maxima"`, `"OnSeparatrixEnergy"`
`NOLTurningPoints[m,E]`	sorted real roots of $V(x)=E$
`NOLBoundedComponent[m,x0,v0]`	$\{x_-,x_+\}$ enclosing x_0, or `Failure`
`NOLPeriod[m,x0,v0]`	period by Eq. (4); `Infinity` at equilibria; `Failure` on separatrices or unbounded motion. Options: `WorkingPrecision`, `PrecisionGoal`, `"Tolerance"` (default 10^{-9})
`NOLPeriodFromEnergy[m,E,x0]`	period of the energy-E orbit through the well containing x_0
`NOLDuffingExactPeriod[a,b,A]`	closed-form $K(m)$ period for $c=0$ and turning point A
`NOLDuffingExactSolution[a,b,A,t]`	$A\,\mathrm{cn}$ or $A\,\mathrm{dn}$ solution with $x(0)=A$, $\dot{x}(0)=0$
`NOLSeparatrixPeriodAsymptotics[m,xs,dE,k]`	leading term $(k/\lambda)\ln(1/\lvert dE\rvert)$, $\lambda=\sqrt{-V''(x_s)}$
`NOLDuffingSolution[p,q,y0,y1]`	closed-form solution of $\ddot{y}+py+qy^3=0$ (case label, modulus, frequency, period or blow-up time, `"y"`)
`NOLExactSolution[m,x0,v0]`	closed form (7); key `"x"` is a pure function of t, plus representation data, or `Failure`
`NOLExactSolutionError[m,x0,v0,sol,tmax]`	$\max\lvert x_{\mathrm{closed}}-x_{\mathrm{ref}}\rvert$ on $[0,t_{\max}]$ against a 32-digit `NDSolve` reference
`NOLSimulate[m,x0,v0,tEnd]`	`<|"x","v","tEnd",...|>` from an 8th-order explicit Runge–Kutta method with dense output; options `WorkingPrecision`, `PrecisionGoal`, `AccuracyGoal`, `Method`
`NOLEnergyDrift[m,sol]`	$\delta_E=\max_j\lvert E_j-E_0\rvert/(1+\lvert E_0\rvert)$ on 2001 samples (option `"Samples"`)
`NOLPhasePortrait[m,{x1,x2},{v1,v2}]`	energy level sets with separatrices in red; option `"Levels"` and all `ContourPlot` options

A.3 Usage examples

The outputs shown in comments are those obtained with Mathematica 15.0; the period values agree with Table 2.

Energy geometry.

```
m = NOLModel[1, -4, 3];                 (* triple well *)
NOLStationaryPoints[m]                  (* {-1, -1/Sqrt[3], 0, 1/Sqrt[3], 1}   exact *)
NOLPotential[m, 1/Sqrt[3]]              (* 2/27   barrier energy *)
NOLClassifyEnergy[m, 1/2, 0]["Maxima"]  (* {{-1/Sqrt[3], 2/27}, {1/Sqrt[3], 2/27}} *)
NOLBoundedComponent[m, 1.1, 0.]         (* {x-, x+} of the side-well orbit *)
NOLPhasePortrait[m, {-1.35, 1.35}, {-0.75, 0.75}, "Levels" -> {0.01, 0.04, 0.07, 0.1, 0.15}]
```

Periods at machine and arbitrary precision.

```
NOLPeriod[NOLModel[1, 0.4, 0.2], 0.8, 0.1]                      (* 5.633261676532774 *)
NOLPeriod[NOLModel[1, 2/5, 1/5], 4/5, 1/10, WorkingPrecision -> 50]  (* 50-digit value *)
{NOLPeriod[NOLModel[-1, 1], 1.2, 0], NOLDuffingExactPeriod[-1, 1, 1.2]}
                                  (* {4.623905221811913, 4.623905221811913} *)
NOLPeriod[NOLModel[-1, 1], 1, 2^(-1/2)]   (* message NOLPeriod::sep, returns Failure *)
NOLPeriod[NOLModel[1, -1], 2., 0]         (* message NOLPeriod::unb, returns Failure *)
```

Exact input is required for high precision: with `NOLModel[1,0.4,0.2]` the result would only carry machine precision, because 0.4 is a machine number; use `NOLModel[1,2/5,1/5]` instead.

Closed-form solution of the cubic–quintic equation.

```
m   = NOLModel[1, 0.4, 0.2];
sol = NOLExactSolution[m, 0.8, 0.1];
sol["Representation"]           (* "II / y: cn: orbit around 0" *)
KeyDrop[sol, {"x", "y"}]        (* A, lambda, mu, Duffing p and q, y(0), y'(0), ... *)
sol["x"][3.]                    (* x(3) from x = A y/Sqrt[1 + lambda y^2 + mu y^4] *)
NOLExactSolutionError[m, 0.8, 0.1, sol, 20]   (* about 3*10^-14 on [0, 20] *)
Plot[sol["x"][t], {t, 0, 30}, Exclusions -> None]
```

`Exclusions -> None` is recommended when plotting the closed form, because the representation contains `Abs` and `Sign`, which `Plot` would otherwise treat as possible discontinuities.

Simulation and energy drift.

```
s = NOLSimulate[NOLModel[-1, 1], 1.6, 0, 100];
NOLEnergyDrift[NOLModel[-1, 1], s]                    (* relative drift on [0, 100] *)
s = NOLSimulate[NOLModel[-1, 1], 1.6, 0, 100, PrecisionGoal -> 14, AccuracyGoal -> 16];
Plot[{s["x"][t], s["v"][t]}, {t, 0, 100}]
```

Separatrix asymptotics.

```
m = NOLModel[-1, 1]; dE = 10^-12;
NOLPeriodFromEnergy[m, -dE, 1, WorkingPrecision -> 40, "Tolerance" -> 10^-30] -
  NOLSeparatrixPeriodAsymptotics[m, 0, dE, 1]        (* -> Log[16] = 2.7725887... *)
```

A smaller `"Tolerance"` is needed here because the default 10^{-9} would classify an energy 10^{-12} below the barrier as lying on the separatrix.

A.4 Package source: `NonlinearOscillatorLab.wl`

The closed-form solver (`duffingSolution`, `cqCandidates`, `cqBuild`, `cqSolution`) is the author's original code, kept verbatim inside the package.

```
(* ::Package:: *)

(* ::Title:: *)
(*NonlinearOscillatorLab - Wolfram Language companion implementation*)


(* ::Text:: *)
(*Version 0.1.0. MIT License. Author: Alvaro H. Salas (ahsalass@unal.edu.co).*)
(*Conservative oscillator  x'' + a x + b x^3 + c x^5 == 0.*)
(**)
(*This package mirrors the Python API (Model, stationary_points, classify_energy,*)
(*turning_points, period, simulate, energy_drift) and adds:*)
(*  - exact / arbitrary-precision arithmetic for every routine;*)
(*  - a singularity-free period integrand obtained by polynomial factorisation*)
(*    of E - V(x) = (x - x_-)(x_+ - x) Q(x);*)
(*  - closed-form Jacobi-elliptic periods for the cubic (Duffing) case, used as an*)
(*    independent analytic reference;*)
(*  - the logarithmic separatrix asymptotics of the period.*)


BeginPackage["NonlinearOscillatorLab`"];


NOLModel::usage =
"NOLModel[a, b, c] represents the conservative oscillator x'' + a x + b x^3 + c x^5 == 0. \
NOLModel[a, b] sets c = 0 and NOLModel[a] sets b = c = 0.";
```

```
NOLPotential::usage = "NOLPotential[model, x] gives V(x) = a x^2/2 + b x^4/4 + c x^6/6.";
NOLForce::usage = "NOLForce[model, x] gives -(a x + b x^3 + c x^5).";
NOLEnergy::usage = "NOLEnergy[model, x, v] gives v^2/2 + V(x).";
NOLCurvature::usage = "NOLCurvature[model, x] gives V''(x) = a + 3 b x^2 + 5 c x^4.";
NOLStationaryPoints::usage =
"NOLStationaryPoints[model] gives the sorted real roots of V'(x) = 0 (exact when the coefficients are exact) .";
NOLClassifyEnergy::usage =
"NOLClassifyEnergy[model, x0, v0] returns an Association with the energy, the equilibria, the \
minima and maxima {x, V(x)}, and whether E coincides with a barrier (separatrix) energy.";
NOLTurningPoints::usage = "NOLTurningPoints[model, E] gives all real solutions of V(x) == E, sorted.";
NOLBoundedComponent::usage =
"NOLBoundedComponent[model, x0, v0] gives {x-, x+}, the consecutive turning points enclosing x0 \
inside which E - V(x) > 0, or a Failure object.";
NOLPeriod::usage =
"NOLPeriod[model, x0, v0] gives the period of the closed orbit through (x0, v0) by quadrature of the \
regularised integrand 2 Integrate[1/Sqrt[2 Q(c + h Sin[t])], {t, -Pi/2, Pi/2}]. Returns Infinity at an \
equilibrium and a Failure object for separatrix or unbounded motion.";
NOLPeriodFromEnergy::usage =
"NOLPeriodFromEnergy[model, E, x0] gives the period of the energy-E orbit through the well that contains x0. ";
NOLDuffingExactPeriod::usage =
"NOLDuffingExactPeriod[a, b, A] gives the closed-form Jacobi-elliptic period of x'' + a x + b x^3 == 0 \
for the orbit with turning point A (cn branch, or dn branch for intra-well orbits of the double well).";
NOLDuffingExactSolution::usage =
"NOLDuffingExactSolution[a, b, A, t] gives the closed-form Jacobi-elliptic solution with x(0) = A, x'(0) = 0.";
NOLSeparatrixPeriodAsymptotics::usage =
"NOLSeparatrixPeriodAsymptotics[model, xs, dE, k] gives the leading term (k/lambda) Log[1/|dE|] of the period \
near a barrier at xs, lambda = Sqrt[-V''(xs)], where k is the number of passages near the saddle per period. ";
NOLSimulate::usage =
"NOLSimulate[model, x0, v0, tEnd] integrates the IVP with an 8th-order explicit Runge-Kutta method \
(the analogue of SciPy's DOP853) and returns an Association with keys \"x\", \"v\", \"tEnd\".";
NOLEnergyDrift::usage =
"NOLEnergyDrift[model, sol] gives max_j |E_j - E_0|/(1 + |E_0|) on a uniform grid of n points (option \"Samples\").";
NOLPhasePortrait::usage = "NOLPhasePortrait[model, {xmin, xmax}, {vmin, vmax}] draws energy level sets and separatrices.";
NOLDuffingSolution::usage =
"NOLDuffingSolution[p, q, y0, y1] gives an Association with the closed-form Jacobi-elliptic (or hyperbolic) \
solution \"y\" of y'' + p y + q y^3 == 0, y(0) = y0, y'(0) = y1, its case label, modulus, frequency and \
period or blow-up time (A. H. Salas).";
NOLExactSolution::usage =
"NOLExactSolution[model, x0, v0] or NOLExactSolution[a, b, c, x0, v0] gives the closed-form solution of \
x'' + a x + b x^3 + c x^5 == 0 in the form x = A y/Sqrt[1 + lambda y^2 + mu y^4], where y solves a cubic \
Duffing equation (A. H. Salas). The result is an Association with key \"x\" (a pure function of t) and the \
representation data, or a Failure object when no real representation is found.";
NOLExactSolutionError::usage =
"NOLExactSolutionError[model, x0, v0, sol, tmax] gives max |x_exact(t) - x_ref(t)| on a uniform grid of [0, tmax], \
where x_ref is a 32-digit NDSolve reference solution.";
NOLVersion::usage = "NOLVersion gives the package version string.";

NOLPeriod::sep = "Initial energy `1` coincides with a separatrix (barrier) energy: the period is infinite.";
NOLPeriod::unb = "Initial state is not in a verified bounded component.";
NOLDuffingExactPeriod::dom = "No bounded periodic orbit with turning point `1` for a = `2`, b = `3`.";
NOLSimulate::args = "tEnd must be positive.";


Begin["`Private`"];


NOLVersion = "0.1.0";

NOLModel[a_] := NOLModel[a, 0, 0];
```

```
NOLModel[a_, b_] := NOLModel[a, b, 0];

NOLPotential[NOLModel[a_, b_, c_], x_] := a x^2/2 + b x^4/4 + c x^6/6;
NOLForce[NOLModel[a_, b_, c_], x_] := -(a x + b x^3 + c x^5);
NOLEnergy[m_NOLModel, x_, v_] := v^2/2 + NOLPotential[m, x];
NOLCurvature[NOLModel[a_, b_, c_], x_] := a + 3 b x^2 + 5 c x^4;


(* ---------- helpers ---------- *)

exactQ[m_NOLModel] := And @@ (Precision[#] === Infinity & /@ List @@ m);

(* real roots of a univariate polynomial, exact when possible;
   inexact roots are Newton-polished (skipped at near-double roots) *)
newtonPolish[poly_, x_, r_] := Module[{d = D[poly, x], z = r, dz},
  Do[dz = d /. x -> z; If[Abs[dz] < 10^-8 (1 + Abs[z]), Break[]]; z = z - (poly /. x -> z)/dz, {3}];
  z];
realRoots[poly_, x_, wp_] := Module[{sol, rts},
  sol = If[Precision[poly] === Infinity,
    Quiet@Solve[poly == 0, x, Reals],
    NSolve[poly == 0, x, Reals, WorkingPrecision -> wp]];
  If[sol === {} || ! ListQ[sol], Return[{}]];
  rts = x /. sol;
  If[Precision[poly] =!= Infinity, rts = newtonPolish[poly, x, #] & /@ rts];
  SortBy[DeleteDuplicates[rts, Abs[N[#1 - #2]] < 10^-10 (1 + Abs[N[#1]]) &], N]];


(* ---------- energy geometry ---------- *)

NOLStationaryPoints[m : NOLModel[a_, b_, c_], opts : OptionsPattern[{WorkingPrecision -> MachinePrecision}]]
    ↪  :=
 Module[{x}, realRoots[a x + b x^3 + c x^5, x, OptionValue[WorkingPrecision]]];

Options[NOLClassifyEnergy] = {"Tolerance" -> 10^-9};
NOLClassifyEnergy[m_NOLModel, x0_, v0_, OptionsPattern[]] :=
 Module[{tol = OptionValue["Tolerance"], en, eq, mx, mn},
  en = NOLEnergy[m, x0, v0];
  eq = NOLStationaryPoints[m];
  mx = {#, NOLPotential[m, #]} & /@ Select[eq, N[NOLCurvature[m, #]] < -tol &];
  mn = {#, NOLPotential[m, #]} & /@ Select[eq, N[NOLCurvature[m, #]] > tol &];
  <|"Energy" -> en, "Equilibria" -> eq, "Minima" -> mn, "Maxima" -> mx,
    "OnSeparatrixEnergy" -> AnyTrue[mx, Abs[N[en - #[[2]]]] <= tol (1 + Abs[N[#[[2]]]]) &]|>];

NOLTurningPoints[m_NOLModel, en_, opts : OptionsPattern[{WorkingPrecision -> MachinePrecision}]] :=
 Module[{x}, realRoots[NOLPotential[m, x] - en, x, OptionValue[WorkingPrecision]]];

Options[NOLBoundedComponent] = {"Tolerance" -> 10^-9, WorkingPrecision -> MachinePrecision};
NOLBoundedComponent[m_NOLModel, x0_, v0_, OptionsPattern[]] :=
 Module[{tol = OptionValue["Tolerance"], en, r, pairs},
  en = NOLEnergy[m, x0, v0];
  r = NOLTurningPoints[m, en, WorkingPrecision -> OptionValue[WorkingPrecision]];
  pairs = Select[Partition[r, 2, 1],
    N[en - NOLPotential[m, Mean[#]]] > tol && N[#[[1]]] - tol <= N[x0] <= N[#[[2]]] + tol &];
  If[Length[pairs] == 1, First[pairs],
   Failure["Unbounded", <|"MessageTemplate" :> NOLPeriod::unb|>]]];


(* ---------- period ---------- *)

(* E - V(x) = (x - xl)(xr - x) Q(x); with x = c + h Sin[t], dx/Sqrt[2(E-V)] = dt/Sqrt[2 Q(x)] *)
regularisedPeriod[m_NOLModel, en_, {xl_, xr_}, wp_, pg_] :=
 Module[{x, t, q, ctr = (xl + xr)/2, h = (xr - xl)/2, f, nodes},
  q = PolynomialQuotient[Expand[en - NOLPotential[m, x]], Expand[(x - xl) (xr - x)], x];
  f = Function[{tt}, Evaluate[1/Sqrt[2 q /. x -> ctr + h Sin[tt]]]];
  (* split where the orbit crosses a stationary point (integrand peaks near separatrices) *)
  nodes = Join[{-Pi/2},
    Sort[ArcSin[(# - ctr)/h] & /@ Select[NOLStationaryPoints[m, WorkingPrecision -> wp], xl < # < xr &]], {
     ↪ Pi/2}];
```

```
  nodes = If[wp === MachinePrecision, N[nodes], SetPrecision[nodes, wp]];
  2 NIntegrate[f[t], Evaluate@Prepend[nodes, t], WorkingPrecision -> wp, PrecisionGoal -> pg,
    Method -> "GaussKronrodRule", MaxRecursion -> 30]];

Options[NOLPeriod] = {"Tolerance" -> 10^-9, WorkingPrecision -> MachinePrecision, PrecisionGoal -> Automatic
    ↪ };
NOLPeriod[m_NOLModel, x0_, v0_, OptionsPattern[]] :=
 Module[{tol = OptionValue["Tolerance"], wp = OptionValue[WorkingPrecision], pg, en, comp},
  pg = Replace[OptionValue[PrecisionGoal], Automatic :> If[wp === MachinePrecision, 13, wp - 6]];
  If[Abs[N[v0]] < tol && Abs[N[NOLForce[m, x0]]] < tol, Return[Infinity]];
  en = NOLEnergy[m, x0, v0];
  If[NOLClassifyEnergy[m, x0, v0, "Tolerance" -> tol]["OnSeparatrixEnergy"],
   Message[NOLPeriod::sep, N[en]];
   Return[Failure["Separatrix", <|"MessageTemplate" -> "Separatrix energy: infinite period", "Energy" -> en
     ↪ |>]]];
  comp = NOLBoundedComponent[m, x0, v0, "Tolerance" -> tol,
    WorkingPrecision -> If[wp === MachinePrecision, wp, wp + 20]];
  If[FailureQ[comp], Message[NOLPeriod::unb]; Return[comp]];
  (* 20 guard digits absorb the cancellation in E - V(x) near separatrices *)
  {en, comp} = If[wp === MachinePrecision, N[{en, comp}], N[{en, comp}, wp + 20]];
  regularisedPeriod[If[wp === MachinePrecision, N[m], N[m, wp + 20]], en, comp, wp, pg]];

NOLPeriodFromEnergy[m_NOLModel, en_, x0_, opts : OptionsPattern[NOLPeriod]] :=
 Module[{v2 = 2 (en - NOLPotential[m, x0])},
  If[N[v2] < 0, Failure["Inaccessible", <|"MessageTemplate" -> "x0 is not accessible at this energy"|>],
   NOLPeriod[m, x0, Sqrt[v2], opts]]];


(* ---------- closed-form Duffing references (c = 0) ---------- *)

NOLDuffingExactPeriod[a_, b_, amp_] :=
 Module[{en = a amp^2/2 + b amp^4/4, w2, mm},
  Which[
   (* intra-well orbit of the double well: x = A dn(w t | m), period 2K/w *)
   N[a] < 0 && N[b] > 0 && N[en] < 0,
    w2 = b amp^2/2; mm = 2 + 2 a/(b amp^2); 2 EllipticK[mm]/Sqrt[w2],
   (* cn branch: x = A cn(w t | m), period 4K/w *)
   N[a + b amp^2] > 0 && N[b amp^2/(2 (a + b amp^2))] < 1 && ! (N[a] < 0 && N[b] > 0 && N[en] == 0),
    w2 = a + b amp^2; mm = b amp^2/(2 w2); 4 EllipticK[mm]/Sqrt[w2],
   True, Message[NOLDuffingExactPeriod::dom, amp, a, b]; $Failed]];

NOLDuffingExactSolution[a_, b_, amp_, t_] :=
 Module[{en = a amp^2/2 + b amp^4/4, w2, mm},
  If[N[a] < 0 && N[b] > 0 && N[en] < 0,
   w2 = b amp^2/2; mm = 2 + 2 a/(b amp^2); amp JacobiDN[Sqrt[w2] t, mm],
   w2 = a + b amp^2; mm = b amp^2/(2 w2); amp JacobiCN[Sqrt[w2] t, mm]]];

NOLSeparatrixPeriodAsymptotics[m_NOLModel, xs_, dE_, k_ : 1] :=
 k/Sqrt[-NOLCurvature[m, xs]] Log[1/Abs[dE]];


(* ---------- closed-form solutions (A. H. Salas) ---------- *)
(* The three routines below are the author's original closed-form solver, kept verbatim. *)

(* ================Cubic Duffing solver for y================*)
duffingSolution[p_?NumericQ, q_?NumericQ, y0_?NumericQ,
   y1_?NumericQ] :=
  Catch[Module[{h, X, dd, roots, pos, A2, A, B2, b2, B, m, w, c1, k,
     R, sg, ua, below, r, s, sn0, a}, h = y1^2 + p y0^2 + q y0^4/2;
    X = y0^2;
    If[q == 0,
     Throw[<|"Case" -> "Linear",
       "y" -> Function[t,
         Evaluate[
          Re[y0 Cos[Sqrt[p + 0. I] t] +
            y1 Sin[Sqrt[p + 0. I] t]/Sqrt[p + 0. I]]]]|>]];
    dd = p^2 + 2 q h;
    roots = If[dd < 0, {}, Sort[{(-p + Sqrt[dd])/q, (-p - Sqrt[dd])/q}]];
```

```
    If[y1 == 0 && roots =!= {},
     roots = ReplacePart[roots,
       First@Ordering[Abs[roots - X], 1] -> X]];
    pos = Select[roots, # > 0 &];
    (*orbit around 0:A cn*)
    If[h > 0 && pos =!= {} && X <= Min[pos], A2 = Min[pos];
     A = Sqrt[A2]; w = p + q A2; m = q A2/(2 w);
     c1 = If[y1 >= 0, -1, 1] InverseJacobiCN[Clip[y0/A, {-1, 1}], m];
     Throw[<|"Case" -> "cn: orbit around 0", "A" -> A, "m" -> m,
       "\[Omega]" -> w, "c1" -> c1, "Period" -> 4 EllipticK[m]/Sqrt[w],
        "y" -> Function[t, Evaluate[A JacobiCN[c1 + Sqrt[w] t, m]]]|>]];
    If[q > 0,(*separatrix*)
     If[h == 0,
      If[y0 == 0,
       Throw[<|"Case" -> "Equilibrium", "y" -> Function[t, 0]|>]];
      a = Sqrt[-2 p/q]; c1 = If[y0 y1 > 0, -1, 1] ArcSech[Abs[y0]/a];
      Throw[<|"Case" -> "Separatrix (sech)",
        "y" -> Function[t, Evaluate[Sign[y0] a Sech[Sqrt[-p] t + c1]]]|>]];
     (*one well:B dn*)B2 = Max[roots]; b2 = Min[roots]; B = Sqrt[B2];
     m = 1 - b2/B2; w = q B2/2;
     c1 =
      If[y0 y1 > 0, -1, 1] InverseJacobiDN[
        Clip[Abs[y0]/B, {Sqrt[1 - m], 1}], m];
     Throw[<|"Case" -> "dn: oscillation inside one well", "B" -> B,
       "m" -> m, "\[Omega]" -> w, "c1" -> c1,
       "Period" -> 2 EllipticK[m]/Sqrt[w],
       "y" -> Function[t,
         Evaluate[Sign[y0] B JacobiDN[c1 + Sqrt[w] t, m]]]|>]];
    (*q<0*)k = -q;
    If[h > 0 && pos === {}, R = Sqrt[2 h/k]; m = 1/2 + p/(2 k R);
     w = 2 k R; sg = If[y1 >= 0, 1, -1];
     c1 = Sign[y0] InverseJacobiCN[(R - X)/(R + X), m];
     Throw[<|"Case" -> "Crosses 0 and blows up", "m" -> m,
       "\[Omega]" -> w, "c1" -> c1,
       "BlowUpTime" -> (2 EllipticK[m] - sg c1)/Sqrt[w],
       "y" -> Function[t,
         Evaluate[
          Sqrt[R] JacobiSN[c1 + sg Sqrt[w] t,
             m]/(1 + JacobiCN[c1 + sg Sqrt[w] t, m])]]|>]];
    ua = Max[Select[pos, # <= X &]];
    below = Sort[Select[Join[roots, {0}], # < ua &]];
    If[Length[below] < 2,
     Throw[Failure["NotCovered", <|"Message" -> "Degenerate case"|>]]];
    r = below[[-1]]; s = below[[-2]];
    m = (r - s)/(ua - s); w = k/2 (ua - s);
    sn0 = Sqrt[Clip[(X - ua)/(X - r), {0, 1}]];
    c1 = If[y0 y1 >= 0, 1, -1] InverseJacobiSN[sn0, m];
    <|"Case" -> "Outside barrier: blows up", "m" -> m,
     "\[Omega]" -> w, "c1" -> c1,
     "BlowUpTime" -> (EllipticK[m] - c1)/Sqrt[w],
     "y" -> Function[t,
       Evaluate[
        Sign[y0] Sqrt[ua - r JacobiSN[c1 + Sqrt[w] t, m]^2]/
          JacobiCN[c1 + Sqrt[w] t, m]]]|>]];

(* ==========Cubic-quintic:x=A y/Sqrt[1+\[Lambda] y^2+\[Mu] y^4]==========*)

cqCandidates[\[Alpha]_, \[Beta]_, \[Gamma]_, x0_, v0_] :=
  Module[{h, X = x0^2, P, roots, out = {}, s, \[Lambda], \[Mu], p, q,
    y1, \[Sigma], pp, j, kl, tol = 10^-9},
   h = v0^2 + \[Alpha] x0^2 + \[Beta] x0^4/2 + \[Gamma] x0^6/3;
   roots =
    P /. NSolve[h P^3 - \[Alpha] P^2 - \[Beta]/2 P - \[Gamma]/3 == 0,
      P];
   If[\[Gamma] == 0,
    roots = Append[P /. NSolve[h P^2 - \[Alpha] P - \[Beta]/2 == 0, P],
       0.]];
   Do[If[Abs[Im[roots[[j]]]] < tol,
```

```
     With[{pj = Re[roots[[j]]]},(*Family I:\[Mu]=0*)
      If[Abs[1 - pj X] > tol, s = 1/(1 - pj X); \[Lambda] = s pj;
       AppendTo[
        out, <|"Family" -> "I", "s" -> s, "\[Lambda]" -> \[Lambda],
         "\[Mu]" -> 0, "p" -> \[Alpha] - 3 h pj,
         "q" -> s (\[Beta] + 4 \[Alpha] pj - 6 h pj^2), "y1" -> s v0|>]];
      (*Family II:\[Mu]\[NotEqual]0*)kl = Delete[roots, j];
      \[Sigma] = Total[kl]; pp = Times @@ kl;
      If[
       Abs[Im[\[Sigma]]] < tol && Abs[Im[pp]] < tol &&
        1 - Re[\[Sigma]] X + Re[pp] X^2 >= 0, \[Sigma] = Re[\[Sigma]];
        pp = Re[pp];
       s = 4/(2 - \[Sigma] X + 2 Sqrt[1 - \[Sigma] X + pp X^2]);
       \[Lambda] = s \[Sigma]/2; \[Mu] = s^2 (\[Sigma]^2 - 4 pp)/16;
       If[Abs[1 - \[Mu] X^2] > tol,
        AppendTo[
         out, <|"Family" -> "II", "s" -> s, "\[Lambda]" -> \[Lambda],
          "\[Mu]" -> \[Mu], "p" -> h (pj - \[Sigma]/2),
          "q" -> -h s (\[Sigma]^2 - 4 pp)/8,
          "y1" -> s v0/(1 - \[Mu] X^2)|>]]]], {j, Length[roots]}];
   out];

cqBuild[\[Alpha]_, \[Beta]_, \[Gamma]_, x0_, v0_, c_] :=
  Module[{ys, yf, xf, signed, h, Pf, ts, yv, Dv, dl = 10^-3, acc, tay,
     dt = 10^-6, xp, xv, res, tw = 6.},
   ys = duffingSolution[c["p"], c["q"], x0, c["y1"]];
   If[FailureQ[ys], Return[$Failed]];
   yf = ys["y"];
   signed =
    StringStartsQ[ys["Case"], "cn"] ||
     StringStartsQ[ys["Case"],
      "Linear"] || (c["\[Lambda]"] == 0 && c["\[Mu]"] == 0);
   xf = If[signed,
     Function[t,
      Evaluate[
       Sqrt[c["s"]] yf[t]/
         Sqrt[1 + c["\[Lambda]"] yf[t]^2 + c["\[Mu]"] yf[t]^4]]],
     Function[t,
      Evaluate[
       Sign[x0] Sqrt[c["s"]] Abs[yf[t]]/
         Sqrt[1 + c["\[Lambda]"] yf[t]^2 + c["\[Mu]"] yf[t]^4]]]];
   If[c["\[Lambda]"] == 0 && c["\[Mu]"] == 0 &&
     KeyExistsQ[ys, "BlowUpTime"], tw = Min[tw, 0.9 ys["BlowUpTime"]]];
   h = v0^2 + \[Alpha] x0^2 + \[Beta] x0^4/2 + \[Gamma] x0^6/3;
   Pf[u_] := h - \[Alpha] u - \[Beta] u^2/2 - \[Gamma] u^3/3;
   ts = Range[tw/400, tw, tw/400];
   yv = Select[Quiet[yf /@ ts], NumericQ[#] && Abs[#] < 10^6 &];
   Dv = 1 + c["\[Lambda]"] yv^2 + c["\[Mu]"] yv^4;
   If[Min[Dv] <= 0, Return[$Failed]];
   If[Abs[xf[0.] - x0] > 10^-8, Return[$Failed]];
   acc = -\[Alpha] x0 - \[Beta] x0^3 - \[Gamma] x0^5;
   tay = x0 + v0 dl + acc dl^2/2;
   If[Abs[tay - x0] > 0 && Abs[xf[dl] - tay] > 0.05 Abs[tay - x0],
    Return[$Failed]];
   res = Quiet@Table[xv = xf[t]; xp = (xf[t + dt] - xf[t - dt])/(2 dt);
      If[NumericQ[xp] && Abs[yf[t]] < 10^6,
       Abs[xp^2 - Pf[xv^2]]/(1 + Abs[Pf[xv^2]]), 0], {t, ts}];
   If[Max[res] > 10^-3, Return[$Failed]];
   <|"Representation" -> c["Family"] <> " / y: " <> ys["Case"],
    "A" -> Sqrt[c["s"]], "\[Lambda]" -> c["\[Lambda]"],
    "\[Mu]" -> c["\[Mu]"], "Duffing p" -> c["p"], "Duffing q" -> c["q"],
     "y(0)" -> x0, "y'(0)" -> c["y1"], "ySolution" -> KeyDrop[ys, "y"],
     "y" -> yf, "x" -> xf,
    "Bounded y" -> ! KeyExistsQ[ys, "BlowUpTime"]|>];

cqSolution[\[Alpha]_?NumericQ, \[Beta]_?NumericQ, \[Gamma]_?NumericQ,
   x0_?NumericQ, v0_?NumericQ] :=
  Catch[Module[{good, h, roots, u, Pf, others, xo, so, tb},
```

```
    good = DeleteCases[
      cqBuild[\[Alpha], \[Beta], \[Gamma], x0, v0, #] & /@
       cqCandidates[\[Alpha], \[Beta], \[Gamma], x0, v0], $Failed];
    If[good =!= {},
     Throw[First[SortBy[good, If[#["Bounded y"], 0, 1] &]]];
    (*start at rest at a turning point that is a pole of every y:
    start from the other end of the motion and shift time*)
    If[v0 == 0, h = \[Alpha] x0^2 + \[Beta] x0^4/2 + \[Gamma] x0^6/3;
     Pf[w_] := h - \[Alpha] w - \[Beta] w^2/2 - \[Gamma] w^3/3;
     roots =
      Cases[u /.
        NSolve[\[Gamma]/3 u^3 + \[Beta]/2 u^2 + \[Alpha] u - h == 0,
         u, Reals], _?NumericQ];
     others =
      SortBy[Select[
        roots, # > 0 && Abs[# - x0^2] > 10^-9 &&
          Pf[(# + x0^2)/2] > 0 &], Abs[# - x0^2] &];
     Do[xo = Sign[x0] Sqrt[uo];
      so =
       DeleteCases[
        cqBuild[\[Alpha], \[Beta], \[Gamma], xo, 0, #] & /@
         cqCandidates[\[Alpha], \[Beta], \[Gamma], xo, 0], $Failed];
      so = Select[so, ! #["Bounded y"] &];
      so = If[so === {}, $Failed, First[so]];
      If[so =!= $Failed, tb = so["ySolution"]["BlowUpTime"];
       Throw[
        Join[so, <|
          "Representation" -> so["Representation"] <> " (time-shifted)",
           "x" -> Function[t, Evaluate[so["x"][t + tb]]]|>]]], {uo,
       others}]];
    Failure[
     "NotCovered", <|
      "Message" -> "No real representation (unbounded motion)"|>]];


NOLDuffingSolution[p_?NumericQ, q_?NumericQ, y0_?NumericQ, y1_?NumericQ] := duffingSolution[p, q, y0, y1];
NOLExactSolution[NOLModel[a_, b_, c_], x0_, v0_] := NOLExactSolution[a, b, c, x0, v0];
NOLExactSolution[a_?NumericQ, b_?NumericQ, c_?NumericQ, x0_?NumericQ, v0_?NumericQ] :=
  cqSolution[N[a], N[b], N[c], N[x0], N[v0]];

Options[NOLExactSolutionError] = {"Samples" -> 400, WorkingPrecision -> 32};
NOLExactSolutionError[m : NOLModel[a_, b_, c_], x0_, v0_, sol_Association, tmax_, OptionsPattern[]] :=
 Module[{z, s, ref, wp = OptionValue[WorkingPrecision], ts, ap, bp, cp},
  {ap, bp, cp} = SetPrecision[{a, b, c}, wp];
  ref = NDSolveValue[{z''[s] + ap z[s] + bp z[s]^3 + cp z[s]^5 == 0,
     z[0] == SetPrecision[x0, wp], z'[0] == SetPrecision[v0, wp]}, z, {s, 0, tmax},
    WorkingPrecision -> wp, PrecisionGoal -> wp - 8, AccuracyGoal -> wp - 8, MaxSteps -> Infinity];
  ts = Subdivide[0, tmax, OptionValue["Samples"]];
  Max[Select[Quiet[Abs[(sol["x"] /@ N[ts]) - N[ref /@ ts]]], NumericQ]]];

(* ---------- simulation ---------- *)

Options[NOLSimulate] = {WorkingPrecision -> MachinePrecision, PrecisionGoal -> 10, AccuracyGoal -> 12,
   Method -> {"ExplicitRungeKutta", "DifferenceOrder" -> 8}, MaxSteps -> Infinity};
NOLSimulate[m_NOLModel, x0_, v0_, tEnd_ : 40, OptionsPattern[]] :=
 Module[{x, v, t, sol},
  If[! (N[tEnd] > 0), Message[NOLSimulate::args]; Return[$Failed]];
  sol = First@NDSolve[{x'[t] == v[t], v'[t] == NOLForce[m, x[t]], x[0] == x0, v[0] == v0},
     {x, v}, {t, 0, tEnd},
     WorkingPrecision -> OptionValue[WorkingPrecision], PrecisionGoal -> OptionValue[PrecisionGoal],
     AccuracyGoal -> OptionValue[AccuracyGoal], Method -> OptionValue[Method],
     MaxSteps -> OptionValue[MaxSteps], InterpolationOrder -> All];
  <|"x" -> (x /. sol), "v" -> (v /. sol), "tEnd" -> tEnd, "Model" -> m, "Initial" -> {x0, v0}|>];

Options[NOLEnergyDrift] = {"Samples" -> 2001};
NOLEnergyDrift[m_NOLModel, sol_Association, OptionsPattern[]] :=
 Module[{ts = Subdivide[0, sol["tEnd"], OptionValue["Samples"] - 1], es, e0},
  es = NOLEnergy[m, sol["x"][#], sol["v"][#]] & /@ ts;
```

```
  e0 = First[es];
  Max[Abs[es - e0]]/(1 + Abs[e0])];


(* ---------- visualisation ---------- *)

Options[NOLPhasePortrait] = Join[{"Levels" -> 18, "SeparatrixStyle" -> Directive[Red, Thick]}, Options[
    ↪ ContourPlot]];
NOLPhasePortrait[m_NOLModel, {x1_, x2_}, {v1_, v2_}, opts : OptionsPattern[]] :=
 Module[{x, v, cls, sepE, base},
  cls = NOLClassifyEnergy[m, 0, 0];
  sepE = N[cls["Maxima"][[All, 2]]];
  base = ContourPlot[NOLEnergy[m, x, v], {x, x1, x2}, {v, v1, v2},
    Evaluate@FilterRules[{opts}, Options[ContourPlot]],
    Contours -> OptionValue["Levels"], ContourShading -> None,
    ContourStyle -> GrayLevel[0.45], FrameLabel -> {"x", "v"}];
  If[sepE === {}, base,
   Show[base, ContourPlot[NOLEnergy[m, x, v], {x, x1, x2}, {v, v1, v2},
     Contours -> sepE, ContourShading -> None, ContourStyle -> OptionValue["SeparatrixStyle"],
     PlotPoints -> 60]]];


End[];
EndPackage[];
```

A.5 Verification suite: `RunTests.wls`

```
#!/usr/bin/env wolframscript
(* ::Package:: *)
(* Automated verification suite for the Wolfram Language companion of NonlinearOscillatorLab.
   Run:  wolframscript -file RunTests.wls
   Writes test_report.txt and test_report.csv next to this script. *)

$dir = DirectoryName[$InputFileName];
Get[FileNameJoin[{$dir, "NonlinearOscillatorLab.wl"}]];
$root = ParentDirectory[$dir];

relErr[x_, y_] := Abs[x - y]/Abs[y];

tests = {
  (* ---- the seven Python regression tests, re-implemented ---- *)
  VerificationTest[Abs[NOLPeriod[NOLModel[4], 0.4, 0] - Pi] < 10^-12, True, TestID -> "harmonic_period"],
  VerificationTest[
   Module[{s = NOLSimulate[NOLModel[4], 0.4, 0, 10]},
    {Max[Abs[s["x"][#] - 0.4 Cos[2 #]] & /@ Subdivide[0., 10., 2000]] < 2*^-8,
     NOLEnergyDrift[NOLModel[4], s] < 2*^-9}], {True, True}, TestID -> "harmonic_solution_and_energy"],
  VerificationTest[NOLStationaryPoints[NOLModel[-1, 1]], {-1, 0, 1}, TestID -> "double_well_equilibria_exact
    ↪ "],
  VerificationTest[NOLTurningPoints[NOLModel[1], 1/2], {-1, 1}, TestID -> "turning_points_exact"],
  VerificationTest[
   {NOLClassifyEnergy[NOLModel[-1, 1], 1, 2^(-1/2)]["OnSeparatrixEnergy"],
    FailureQ[Quiet@NOLPeriod[NOLModel[-1, 1], 1, 2^(-1/2)]]}, {True, True}, TestID -> "separatrix_detection"
    ↪ ],
  VerificationTest[
   Module[{m = NOLModel[1, 0.4, 0.2], s},
    s = NOLSimulate[m, 0.8, 0.1, 20];
    {NOLEnergyDrift[m, s] < 5*^-8, NOLPeriod[m, 0.8, 0.1] > 0}], {True, True}, TestID -> "cubic_quintic"],
  VerificationTest[NOLPeriod[NOLModel[1], 0, 0], Infinity, TestID -> "equilibrium_period"],

  (* ---- new: arbitrary precision ---- *)
  VerificationTest[Abs[NOLPeriod[NOLModel[4], 2/5, 0, WorkingPrecision -> 40] - Pi] < 10^-30, True,
   TestID -> "harmonic_period_40_digits"],
  VerificationTest[
   Module[{hp = NOLPeriod[NOLModel[1, 2/5, 1/5], 4/5, 1/10, WorkingPrecision -> 40]},
    {Precision[hp] >= 30, relErr[NOLPeriod[NOLModel[1, 0.4, 0.2], 0.8, 0.1], hp] < 10^-13}], {True, True},
   TestID -> "cubic_quintic_machine_vs_40_digits"],
```

```
38
39  (* ---- new: independent closed-form (Jacobi elliptic) references ---- *)
40  VerificationTest[relErr[NOLPeriod[NOLModel[1, 1], 1., 0], NOLDuffingExactPeriod[1, 1, 1.]] < 10^-13, True,
41   TestID -> "elliptic_hardening"],
42  VerificationTest[relErr[NOLPeriod[NOLModel[1, -1/2], 0.9, 0], NOLDuffingExactPeriod[1, -1/2, 0.9]] <
      ↪ 10^-13, True,
43   TestID -> "elliptic_softening"],
44  VerificationTest[relErr[NOLPeriod[NOLModel[-1, 1], 1.2, 0], NOLDuffingExactPeriod[-1, 1, 1.2]] < 10^-13,
      ↪ True,
45   TestID -> "elliptic_double_well_inner_dn"],
46  VerificationTest[relErr[NOLPeriod[NOLModel[-1, 1], 1.6, 0], NOLDuffingExactPeriod[-1, 1, 1.6]] < 10^-13,
      ↪ True,
47   TestID -> "elliptic_double_well_outer_cn"],
48  VerificationTest[
49   Module[{p = NOLPeriod[NOLModel[-1, 1], 12/10, 0, WorkingPrecision -> 50],
50     q = N[NOLDuffingExactPeriod[-1, 1, 12/10], 50]}, Abs[p - q] < 10^-40], True,
51   TestID -> "elliptic_double_well_50_digits"],
52  VerificationTest[
53   Module[{t, res},
54    res = Table[Module[{xs = NOLDuffingExactSolution[a, b, amp, t]},
55       Quiet[N[(D[xs, {t, 2}] + a xs + b xs^3) /. t -> 7/10, 30], {N::meprec, General::stop}]],
56      {p, {{1, 1, 1}, {1, -1/2, 9/10}, {-1, 1, 6/5}, {-1, 1, 8/5}}}, {a, {p[[1]]}}, {b, {p[[2]]}}, {amp, {p
      ↪ [[3]]}}];
57    Max[Abs[Flatten[res]]] < 10^-25], True, TestID -> "elliptic_solutions_satisfy_ODE"],
58  VerificationTest[
59   Module[{s = NOLSimulate[NOLModel[-1, 1], 1.2, 0, 20]},
60    Max[Abs[s["x"][#] - NOLDuffingExactSolution[-1, 1, 1.2, #]] & /@ Subdivide[0., 20., 400]] < 10^-8], True
      ↪ ,
61   TestID -> "simulation_vs_elliptic_trajectory"],
62
63  (* ---- new: geometry of the cubic-quintic triple well ---- *)
64  VerificationTest[NOLStationaryPoints[NOLModel[1, -4, 3]], {-1, -1/Sqrt[3], 0, 1/Sqrt[3], 1},
65   TestID -> "triple_well_equilibria_exact"],
66  VerificationTest[Simplify[NOLPotential[NOLModel[1, -4, 3], 1/Sqrt[3]]], 2/27, TestID -> "
      ↪ triple_well_barrier"],
67
68  (* ---- new: period/trajectory consistency ---- *)
69  VerificationTest[
70   Module[{m = NOLModel[1, 0.4, 0.2], T, s},
71    T = NOLPeriod[m, 0.8, 0.1]; s = NOLSimulate[m, 0.8, 0.1, 1.5 T];
72    Norm[{s["x"][T] - 0.8, s["v"][T] - 0.1}] < 10^-8], True, TestID -> "period_closes_orbit"],
73  VerificationTest[FailureQ[Quiet@NOLPeriod[NOLModel[1, -1], 2., 0]], True, TestID -> "unbounded_detected"],
74
75  (* ---- new: separatrix asymptotics T ~ (1/lambda) Log[1/dE] + C ---- *)
76  VerificationTest[
77   Module[{m = NOLModel[-1, 1], dE = 10^-10, ti, to},
78    ti = NOLPeriodFromEnergy[m, -dE, 1, WorkingPrecision -> 40, "Tolerance" -> 10^-30];
79    to = NOLPeriodFromEnergy[m, dE, 0, WorkingPrecision -> 40, "Tolerance" -> 10^-30];
80    {Abs[ti - NOLSeparatrixPeriodAsymptotics[m, 0, dE, 1] - Log[16]] < 10^-6,
81     Abs[to - NOLSeparatrixPeriodAsymptotics[m, 0, dE, 2] - 2 Log[16]] < 10^-6}], {True, True},
82   TestID -> "separatrix_log_asymptotics"],
83
84  (* ---- new: closed-form solutions x = A y/Sqrt[1 + lambda y^2 + mu y^4] (A. H. Salas) ---- *)
85  Sequence @@ Table[With[{cs = cs},
86     VerificationTest[
87      Module[{m = NOLModel @@ cs[[2 ;; 4]], sol},
88       sol = NOLExactSolution[m, cs[[5]], cs[[6]]];
89       If[FailureQ[sol], sol, NOLExactSolutionError[m, cs[[5]], cs[[6]], sol, 20] < 10^-8]], True,
90      TestID -> "closed_form_" <> cs[[1]]]],
91    {cs, {{"hardening", 1, 1, 0, 1., 0.}, {"softening", 1, -1/2, 0, 0.9, 0.},
92      {"dw_inner", -1, 1, 0, 1.2, 0.}, {"dw_outer", -1, 1, 0, 1.6, 0.},
93      {"cq_demo", 1, 0.4, 0.2, 0.8, 0.1}, {"tw_center", 1, -4, 3, 0.3, 0.},
94      {"tw_side", 1, -4, 3, 1.1, 0.}, {"tw_outer", 1, -4, 3, 1.2, 0.3}}}],
95  VerificationTest[
96   Module[{d = NOLDuffingSolution[1., 1., 1., 0.]}, relErr[d["Period"], NOLDuffingExactPeriod[1, 1, 1.]] <
      ↪ 10^-14],
97   True, TestID -> "closed_form_duffing_period"],
98
```

```
  (* ---- new: cross-implementation agreement with the Python package ---- *)
  VerificationTest[
   Module[{rows = Rest@Import[FileNameJoin[{$root, "benchmarks", "python_periods.csv"}], "CSV"]},
    Max[Table[relErr[NOLPeriod[NOLModel @@ r[[2 ;; 4]], r[[5]], r[[6]]], r[[7]]], {r, rows}]] < 10^-9], True
    ,
   TestID -> "python_cross_validation"]
};

report = TestReport[tests];
lines = Table[{t["TestID"], ToString[t["Outcome"]], Round[1000 QuantityMagnitude[t["AbsoluteTimeUsed"], "
    Seconds"], 0.1]},
   {t, Values[report["TestResults"]]}];
Export[FileNameJoin[{$dir, "test_report.csv"}], Prepend[lines, {"test", "outcome", "ms"}], "CSV"];
summary = StringJoin[
   "NonlinearOscillatorLab (Wolfram Language) ", NOLVersion, " - ", $Version, "\n",
   "Passed ", ToString[report["TestsSucceededCount"]], " / ", ToString[Length[tests]], "\n\n",
   StringRiffle[StringRiffle[ToString /@ #, "  "] & /@ lines, "\n"], "\n"];
Export[FileNameJoin[{$dir, "test_report.txt"}], summary, "Text"];
Print[summary];
Do[If[t["Outcome"] =!= "Success", Print[t["TestID"], ": ", t["ActualOutput"]]], {t, Values[report["
    TestResults"]]}];
```

A.6 Figures and tables: `MakeFigures.wls` and `RunAll.wls`

```
#!/usr/bin/env wolframscript
(* ::Package:: *)
(* Publication figures and tables for NonlinearOscillatorLab, generated with the Wolfram Language.
   Run:  wolframscript -file MakeFigures.wls
   Output: ../figures/*.pdf, ../figures/*.png, ../tables/*.csv, ../tables/*.tex *)

$dir = DirectoryName[$InputFileName];
Get[FileNameJoin[{$dir, "NonlinearOscillatorLab.wl"}]];
$root = ParentDirectory[$dir];
$fig = FileNameJoin[{$root, "figures"}]; $tab = FileNameJoin[{$root, "tables"}];
Quiet[CreateDirectory /@ {$fig, $tab}];
$t0 = AbsoluteTime[];
log[s__] := Print[Round[AbsoluteTime[] - $t0, 0.1], "s  ", s];

(* ---------------- style ---------------- *)
cols = RGBColor /@ {"#1f5fa8", "#d1661a", "#2a8c55", "#7a4fa3", "#c0392b", "#5d6d7e", "#b8860b", "#17becf"};
font = {FontFamily -> "Times New Roman", FontSize -> 11};
frameOpts = Sequence[Frame -> True, FrameStyle -> Directive[Black, AbsoluteThickness[0.7]],
   FrameTicksStyle -> Directive[Black, 10], BaseStyle -> font, ImageSize -> 330, AspectRatio -> 0.72,
   ImagePadding -> {{52, 12}, {40, 10}}];
lbl[g_, s_String] := Labeled[g, Style[s, Bold, 12, FontFamily -> "Times New Roman"], {{Top, Left}}];
row[gs__] := Grid[{{gs}}, Spacings -> {1.5, 0}, Alignment -> Top];
save[name_, g_] := (Export[FileNameJoin[{$fig, name <> ".pdf"}], g];
   Export[FileNameJoin[{$fig, name <> ".png"}], g, ImageResolution -> 300]; log["wrote ", name]);
(* legends always below the frame, two entries per row: never on top of data *)
leg[cs_, ls_, ___] := Placed[LineLegend[cs, ls, LegendMarkerSize -> {20, 10}, LabelStyle -> Append[font,
    10],
    LegendLayout -> {"Row", 2}], Below];
ptleg[cs_, ls_, ___] := Placed[PointLegend[cs, ls, LegendMarkerSize -> 8, LabelStyle -> Append[font, 10],
    LegendLayout -> {"Row", 2}], Below];
num[x_String] := If[StringMatchQ[x, NumberString ~~ ("e" | "E") ~~ ("+" | "-" | "") ~~ DigitCharacter ..],
   ToExpression[StringReplace[x, {"e" -> "*^", "E" -> "*^"}]], x];
num[x_?NumericQ] := x;
eqMarks[m_, col_] := Module[{c = NOLClassifyEnergy[m, 0, 0]},
   {col, PointSize[0.018], Point[N@c["Minima"]], White, EdgeForm[{col, AbsoluteThickness[1.2]}],
    Disk[#, Offset[3.2]] & /@ N@c["Maxima"]}];

(* ---------------- Fig. 1: potentials and equilibria ---------------- *)
duff = {NOLModel[1, 1], NOLModel[1, -1/2], NOLModel[-1, 1]};
cq = {NOLModel[1, 2/5, 1/5], NOLModel[1, -4, 3]};
f1a = Plot[Evaluate[NOLPotential[#, x] & /@ duff], {x, -2.3, 2.3}, PlotRange -> {-0.4, 1.6},
   PlotStyle -> (Directive[#, AbsoluteThickness[1.6]] & /@ cols[[1 ;; 3]]),
```

```
   Epilog -> MapThread[eqMarks, {duff, cols[[1 ;; 3]]}], FrameLabel -> {"x", "V(x)"},
   PlotLegends -> leg[cols[[1 ;; 3]], {"hardening a=1, b=1", "softening a=1, b=-1/2", "double well a=-1, b=1
     ↪ "}, Scaled[{0.5, 0.86}]],
   Evaluate[frameOpts]];
f1b = Plot[Evaluate[NOLPotential[#, x] & /@ cq], {x, -1.4, 1.4}, PlotRange -> {-0.03, 0.32},
   PlotStyle -> (Directive[#, AbsoluteThickness[1.6]] & /@ cols[[4 ;; 5]]),
   Epilog -> MapThread[eqMarks, {cq, cols[[4 ;; 5]]}], FrameLabel -> {"x", "V(x)"},
   PlotLegends -> leg[cols[[4 ;; 5]], {"a=1, b=2/5, c=1/5", "triple well a=1, b=-4, c=3"}, Scaled[{0.5,
     ↪ 0.88}]],
   Evaluate[frameOpts]];
save["fig1_potentials", row[lbl[f1a, "(a) cubic (Duffing)"], lbl[f1b, "(b) cubic\[Dash]quintic"]]];

(* ---------------- Fig. 2: phase portraits ---------------- *)
f2a = NOLPhasePortrait[NOLModel[-1, 1], {-2, 2}, {-1.6, 1.6},
   "Levels" -> DeleteCases[Range[-0.22, 1.3, 0.08], _?(Abs[#] < 0.01 &)], PlotPoints -> 60,
   Epilog -> eqMarks[NOLModel[-1, 1], Black] /. {x_?NumericQ, y_?NumericQ} :> {x, 0},
   Evaluate[frameOpts]];
f2b = NOLPhasePortrait[NOLModel[1, -4, 3], {-1.35, 1.35}, {-0.75, 0.75},
   "Levels" -> {0.005, 0.02, 0.04, 0.06, 0.09, 0.13, 0.18, 0.24}, PlotPoints -> 70,
   Epilog -> eqMarks[NOLModel[1, -4, 3], Black] /. {x_?NumericQ, y_?NumericQ} :> {x, 0},
   Evaluate[frameOpts]];
save["fig2_phase_portraits", row[lbl[f2a, "(a) double well, a=-1, b=1"], lbl[f2b, "(b) triple well, a=1, b
     ↪ =-4, c=3"]]];

(* ---------------- Fig. 3: closed form vs numerics ---------------- *)
ref32[m : NOLModel[a_, b_, c_], x0_, v0_, tmax_] := Module[{z, s},
   NDSolveValue[{z''[s] + SetPrecision[a, 32] z[s] + SetPrecision[b, 32] z[s]^3 + SetPrecision[c, 32] z[s]^5
     ↪  == 0,
    z[0] == SetPrecision[x0, 32], z'[0] == SetPrecision[v0, 32]}, z, {s, 0, tmax},
    WorkingPrecision -> 32, PrecisionGoal -> 24, AccuracyGoal -> 24, MaxSteps -> Infinity]];
demo = {NOLModel[1, 0.4, 0.2], 0.8, 0.1};
exDemo = NOLExactSolution @@ demo;
log["demo representation: ", exDemo["Representation"]];
refDemo = ref32[demo[[1]], demo[[2]], demo[[3]], 30];
simDemo = NOLSimulate[demo[[1]], demo[[2]], demo[[3]], 30];
pyOrbit = Rest@Import[FileNameJoin[{$root, "examples", "orbit.csv"}], "CSV"];
pyOrbit = Map[num, pyOrbit, {2}];
f3a = Legended[Show[
   Plot[exDemo["x"][t], {t, 0, 30}, PlotStyle -> Directive[cols[[1]], AbsoluteThickness[1.6]], PlotPoints ->
     ↪  400,
    Exclusions -> None],
   ListPlot[{pyOrbit[[1 ;; -1 ;; 60, {1, 2}]], Table[{t, simDemo["x"][t]}, {t, 1.5, 30, 3.}]},
    PlotStyle -> {cols[[2]], cols[[3]]},
    PlotMarkers -> {{"\[FilledCircle]", 7}, {"\[EmptySquare]", 9}}],
   FrameLabel -> {"t", "x(t)"}, PlotRange -> {-1.05, 1.05}, Evaluate[frameOpts]],
  Placed[Row[{LineLegend[{Directive[cols[[1]], AbsoluteThickness[1.6]]}, {"closed form (Salas)"}, LabelStyle
     ↪  -> Append[font, 10]],
    PointLegend[{cols[[2]], cols[[3]]}, {"Python DOP853", "Wolfram RK8"}, LegendMarkers -> {"\[FilledCircle
     ↪ ]", "\[EmptySquare]"},
     LegendLayout -> "Row", LabelStyle -> Append[font, 10]]}, "  "], Below]];
errCurve[f_] := Table[{t, Max[Abs[f[t] - refDemo[t]], 10^-17]}, {t, 0.05, 30, 0.05}];
pyInterp = Interpolation[pyOrbit[[All, {1, 2}]], InterpolationOrder -> 5];
f3b = ListLogPlot[{errCurve[exDemo["x"]], errCurve[simDemo["x"]],
    Table[{pyOrbit[[k, 1]], Max[Abs[pyOrbit[[k, 2]] - refDemo[pyOrbit[[k, 1]]]], 10^-17]}, {k, 2, Length[
     ↪ pyOrbit], 5}]},
   Joined -> True, PlotStyle -> (Directive[#, AbsoluteThickness[1.1]] & /@ cols[[1 ;; 3]]),
   PlotRange -> {10^-17, 10^-6}, FrameLabel -> {"t", "|x(t) - x_ref(t)|"},
   PlotLegends -> leg[cols[[1 ;; 3]], {"closed form", "Wolfram RK8", "Python DOP853"}, Scaled[{0.75, 0.2}]],
   Evaluate[frameOpts]];
cases3 = {{"tw_side", NOLModel[1, -4, 3], 1.1, 0.}, {"tw_outer", NOLModel[1, -4, 3], 1.2, 0.3},
   {"dw_inner", NOLModel[-1, 1], 1.2, 0.}};
ex3 = NOLExactSolution[#[[2]], #[[3]], #[[4]]] & /@ cases3;
f3c = Show[
   Plot[Evaluate[#["x"][t] & /@ ex3], {t, 0, 20}, PlotPoints -> 400, Exclusions -> None,
    PlotStyle -> (Directive[#, AbsoluteThickness[1.5]] & /@ cols[[{4, 5, 6}]]),
    PlotLegends -> leg[cols[[{4, 5, 6}]], {"triple well, side", "triple well, above barrier", "double well,
     ↪ intra-well"}, Scaled[{0.5, 0.12}]]],
```

```
   ListPlot[Table[With[{r = ref32[c[[2]], c[[3]], c[[4]], 20]}, Table[{t, r[t]}, {t, 0.5, 20, 1}]], {c, cases3}],
    PlotStyle -> Black, PlotMarkers -> {{"\[EmptyCircle]", 7}}],
   FrameLabel -> {"t", "x(t)"}, PlotRange -> {-1.6, 1.6},
   Evaluate[frameOpts]];
refs3 = Table[ref32[c[[2]], c[[3]], c[[4]], 20], {c, cases3}];
f3d = ListLogPlot[Table[Table[{t, Max[Abs[ex3[[k]]["x"][t] - refs3[[k]][t]], 10^-17]}, {t, 0.05, 20, 0.05}], {k, 3}],
   Joined -> True, PlotStyle -> (Directive[#, AbsoluteThickness[1.1]] & /@ cols[[{4, 5, 6}]]),
   PlotRange -> {10^-17, 10^-6}, FrameLabel -> {"t", "|x(t) - x_ref(t)|"},
   PlotLegends -> leg[cols[[{4, 5, 6}]], {"triple well, side", "triple well, above barrier", "double well, intra-well"}],
   Evaluate[frameOpts]];
save["fig3_closed_form_vs_numerics",
  Grid[{{lbl[f3a, "(a) a=1, b=0.4, c=0.2, x0=0.8, v0=0.1"], lbl[f3b, "(b) pointwise error vs 32-digit reference"]},
    {lbl[f3c, "(c) closed form (lines) vs 32-digit NDSolve (circles)"], lbl[f3d, "(d) pointwise error of the closed form in (c)"]}},
   Spacings -> {1.5, 1.5}, Alignment -> Top]];
save["fig3a_demo_orbit", f3a];

(* ---------------- Fig. 4: randomised campaign of the closed-form solver ---------------- *)
SeedRandom[20261019];
nCampaign = 200; tCampaign = 20;
campaign = Table[
   Module[{a = RandomReal[{-12, 12}], b = RandomReal[{-12, 12}], c = RandomReal[{0, 1}],
     x0 = RandomReal[{-2, 0}], v0 = RandomReal[{-2, 2}], sol, err, tm},
    {tm, sol} = AbsoluteTiming[Quiet@NOLExactSolution[a, b, c, x0, v0]];
    err = If[FailureQ[sol], Missing["Failure"],
      Quiet@NOLExactSolutionError[NOLModel[a, b, c], x0, v0, sol, tCampaign, "Samples" -> 200, WorkingPrecision -> 28]];
    <|"a" -> a, "b" -> b, "c" -> c, "x0" -> x0, "v0" -> v0, "E" -> NOLEnergy[NOLModel[a, b, c], x0, v0],
     "ok" -> ! FailureQ[sol], "rep" -> If[FailureQ[sol], "failure", sol["Representation"]],
     "family" -> If[FailureQ[sol], "failure", First@StringSplit[sol["Representation"], " "]],
     "err" -> err, "time" -> tm|>], {k, nCampaign}];
log["campaign done"];
Export[FileNameJoin[{$tab, "closed_form_campaign.csv"}],
  Prepend[Values /@ (KeyDrop[#, {}] & /@ campaign), Keys[First[campaign]]], "CSV"];
good = Select[campaign, #["ok"] && NumericQ[#["err"]] &];
errs = #["err"] & /@ good;
fams = Union[#["family"] & /@ good];
famCol = AssociationThread[fams, cols[[1 ;; Length[fams]]]];
f4a = Histogram[Log10[Clip[errs, {10^-16, Infinity}]], {-16, Max[-6, Ceiling[Max[Log10[errs]]]], 0.5},
   ChartStyle -> Directive[cols[[1]], EdgeForm[White]], FrameLabel -> {"log10 max|x_closed - x_ref| on [0, 20]", "cases"},
   Evaluate[frameOpts]];
f4b = ListLogPlot[Table[{#["E"], Max[#["err"], 10^-16]} & /@ Select[good, #["family"] == f &], {f, fams}],
   PlotStyle -> (Directive[famCol[#], PointSize[0.012]] & /@ fams), FrameLabel -> {"energy E", "max error"},
   PlotLegends -> ptleg[famCol /@ fams, ("family " <> # & /@ fams), Scaled[{0.8, 0.85}]],
   Evaluate[frameOpts]];
save["fig4_closed_form_campaign", row[lbl[f4a, "(a) error distribution, " <> ToString[Length[good]] <> "/" <> ToString[nCampaign] <> " cases"],
   lbl[f4b, "(b) error versus energy"]]];
campaignSummary = <|"cases" -> nCampaign, "represented" -> Count[campaign, _?(#["ok"] &)],
   "median_error" -> Median[errs], "p95_error" -> Quantile[errs, 0.95], "max_error" -> Max[errs],
   "fraction_below_1e-8" -> N[Count[errs, e_ /; e < 10^-8]/nCampaign],
   "median_time_s" -> Median[#["time"] & /@ campaign],
   "representations" -> Counts[#["rep"] & /@ campaign]|>;
Export[FileNameJoin[{$tab, "closed_form_campaign_summary.txt"}], ToString[campaignSummary, InputForm], "Text"];
log["campaign summary ", campaignSummary];

(* ---------------- Fig. 5: period-energy curves ---------------- *)
pe = Rest@Import[FileNameJoin[{$root, "benchmarks", "python_period_energy.csv"}], "CSV"];
peLegacy = Rest@Import[FileNameJoin[{$root, "benchmarks", "python_period_energy_legacy_v010.csv"}], "CSV"];
pe = Map[num, pe, {2}]; peLegacy = Map[num, peLegacy, {2}];
families = DeleteDuplicates[pe[[All, 1]]];
famData = AssociationMap[Function[f, Select[pe, #[[1]] === f &]], families];
```

```
exactCoeffs = <|"hardening" -> {1, 1, 0}, "softening" -> {1, -1/2, 0}, "dw_inner" -> {-1, 1, 0}, "dw_outer" -> {-1, 1, 0},
   "cq_demo" -> {1, 2/5, 1/5}, "tw_center" -> {1, -4, 3}, "tw_side" -> {1, -4, 3}, "tw_outer" -> {1, -4, 3}|>;
(* machine-precision Wolfram periods and 40-digit references at the same (binary) energies *)
wlT = AssociationMap[Function[f, Table[Quiet[NOLPeriodFromEnergy[NOLModel @@ N[exactCoeffs[f]], r[[6]], r[[5]]],
      {NIntegrate::slwcon, NIntegrate::eincr}], {r, famData[f]}]], families];
log["machine periods done"];
refT = AssociationMap[Function[f, Table[
      Module[{m = NOLModel @@ exactCoeffs[f], en = SetPrecision[r[[6]], 60], x0 = Rationalize[r[[5]]], amp},
       If[exactCoeffs[f][[3]] === 0,
        (* turning point of the orbit through x0 (not the outer root beyond a barrier) *)
        amp = Last@NOLBoundedComponent[m, x0, Sqrt[2 (en - NOLPotential[m, x0])], WorkingPrecision -> 40, "Tolerance" -> 10^-30];
        NOLDuffingExactPeriod[exactCoeffs[f][[1]], exactCoeffs[f][[2]], amp],
        NOLPeriodFromEnergy[m, en, x0, WorkingPrecision -> 40, "Tolerance" -> 10^-30]]], {r, famData[f]}]], families];
log["reference periods done"];
duffF = {"hardening", "softening", "dw_inner", "dw_outer"}; cqF = {"cq_demo", "tw_center", "tw_side", "tw_outer"};
peNames = <|"hardening" -> "hardening", "softening" -> "softening", "dw_inner" -> "double well, inner", "dw_outer" -> "double well, outer",
   "cq_demo" -> "a=1, b=2/5, c=1/5", "tw_center" -> "triple well, centre", "tw_side" -> "triple well, side",  "tw_outer" -> "triple well, outer"|>;
pePlot[fs_, cs_, ttl_] := Show[
   ListLinePlot[Table[Transpose[{famData[f][[All, 6]], wlT[f]}], {f, fs}], PlotStyle -> (Directive[#, AbsoluteThickness[1.4]] & /@ cs)],
   ListPlot[Table[Transpose[{famData[f][[All, 6]], N[refT[f]]}][[1 ;; -1 ;; 3]], {f, fs}],
    PlotStyle -> cs, PlotMarkers -> {{"\[EmptyCircle]", 6}}],
   FrameLabel -> {"energy E", "period T(E)"}, PlotRange -> All, Evaluate[frameOpts]];
f5a = pePlot[duffF, cols[[1 ;; 4]], ""]; f5b = pePlot[cqF, cols[[5 ;; 8]], ""];
f5a = Legended[Show[f5a, PlotRange -> {{-0.3, 5}, {0, 16}}], leg[cols[[1 ;; 4]], peNames /@ duffF]];
f5b = Legended[Show[f5b, PlotRange -> {{0, 1.2}, {0, 20}}], leg[cols[[5 ;; 8]], peNames /@ cqF]];
save["fig5_period_energy", row[lbl[f5a, "(a) Duffing: quadrature (lines), elliptic K (circles)"],
   lbl[f5b, "(b) cubic\[Dash]quintic: quadrature, 40-digit reference"]]];

(* ---------------- Fig. 6: accuracy of the period and separatrix asymptotics ---------------- *)
relE[a_, b_] := Max[N[Abs[a - b]/Abs[b]], 10^-17];
legacyByF = AssociationMap[Function[f, Select[peLegacy, #[[1]] === f &][[All, 7]]], families];
strip[k_, vals_, off_, col_, mk_] := Table[{k + off + RandomReal[{-0.08, 0.08}], Log10[v]}, {v, vals}];
SeedRandom[1];
f6a = ListPlot[
   Join[
    {Flatten[Table[strip[k, MapThread[relE, {legacyByF[families[[k]]], refT[families[[k]]]}], -0.28, 0, 0], {k, 8}], 1]},
    {Flatten[Table[strip[k, MapThread[relE, {famData[families[[k]]][[All, 7]], refT[families[[k]]]}], 0, 0, 0], {k, 8}], 1]},
    {Flatten[Table[strip[k, MapThread[relE, {wlT[families[[k]]], refT[families[[k]]]}], 0.28, 0, 0], {k, 8}], 1]}],
   PlotStyle -> {Directive[cols[[6]], PointSize[0.008]], Directive[cols[[2]], PointSize[0.008]], Directive[cols[[1]], PointSize[0.008]]},
   FrameTicks -> {{Automatic, None}, {Table[{k, Rotate[Style[peNames[families[[k]]], 8], 30 Degree]}, {k, 8}], None}},
   FrameLabel -> {"", "log10 relative error of T"}, PlotRange -> {{0.5, 8.5}, {-17, -4}},
   PlotLegends -> ptleg[cols[[{6, 2, 1}]], {"Python v0.1.0 (sin substitution)", "Python, factored integrand" , "Wolfram, factored integrand"}, Scaled[{0.62, 0.87}]],
   ImagePadding -> {{52, 12}, {70, 10}}, Evaluate[frameOpts]];
sepData = Table[Module[{m = NOLModel[-1, 1], dE = 10^-k},
    {k, NOLPeriodFromEnergy[m, -dE, 1, WorkingPrecision -> 40, "Tolerance" -> 10^-40] - Log[1/dE],
     NOLPeriodFromEnergy[m, dE, 0, WorkingPrecision -> 40, "Tolerance" -> 10^-40] - 2 Log[1/dE]}], {k, Range[1, 16, 1/2]}];
log["separatrix data ", N[sepData]];
sepData = N[sepData];
sepIn = Select[Transpose[{10.^-sepData[[All, 1]], sepData[[All, 2]]}], VectorQ[#, NumericQ] && Im[#[[2]]] == 0 &];
sepOut = Select[Transpose[{10.^-sepData[[All, 1]], sepData[[All, 3]]}], VectorQ[#, NumericQ] && Im[#[[2]]] == 0 &];
```

```
sepIn = Select[Transpose[{-sepData[[All, 1]], sepData[[All, 2]]}], VectorQ[#, NumericQ] && Im[#[[2]]] == 0 &];
sepOut = Select[Transpose[{-sepData[[All, 1]], sepData[[All, 3]]}], VectorQ[#, NumericQ] && Im[#[[2]]] == 0 &];
f6b = Show[
   ListPlot[{sepIn, sepOut}, Joined -> True, PlotMarkers -> {{"\[FilledCircle]", 6}, {"\[FilledSquare]", 6}}, PlotStyle -> cols[[{3, 4}]]],
   Plot[{Log[16], 2 Log[16]}, {x, -16.5, -0.5}, PlotStyle -> Directive[Gray, Dashed]],
   FrameLabel -> {"log10 |E - E_s|", "T - k ln(1/|E - E_s|)"}, PlotRange -> {{-16.5, -0.5}, {2, 6.2}},
   Epilog -> {Text[Style["inner orbit, k=1  \[RightArrow] ln 16", 10, cols[[3]], FontFamily -> "Times New Roman"], Scaled[{0.55, 0.3}]],
    Text[Style["outer orbit, k=2  \[RightArrow] 2 ln 16", 10, cols[[4]], FontFamily -> "Times New Roman"], Scaled[{0.55, 0.72}]]},
   Evaluate[frameOpts]];
save["fig6_period_accuracy", row[lbl[f6a, "(a) relative period error (8 families \[Times] 40 energies)"],
   lbl[f6b, "(b) separatrix asymptotics, double well"]]];

(* ---------------- Fig. 7: arbitrary precision and integrator tolerance ---------------- *)
tHi = NOLPeriod[NOLModel[1, 2/5, 1/5], 4/5, 1/10, WorkingPrecision -> 80];
(* correct digits: compare the digits returned at precision wp (taken at face value) with an 80-digit value *)
precData = Table[Module[{tm, tw},
     {tm, tw} = AbsoluteTiming[NOLPeriod[NOLModel[1, 2/5, 1/5], 4/5, 1/10, WorkingPrecision -> wp]];
     {wp, N[-Log10[Max[Abs[SetPrecision[tw, 120] - SetPrecision[tHi, 120]], 10^-90]]], tm}], {wp, 16, 72, 4}];
log["precision study ", precData];
f7a = Show[ListPlot[precData[[All, {1, 2}]], Joined -> True, PlotMarkers -> {{"\[FilledCircle]", 7}}, PlotStyle -> cols[[1]]],
   Plot[x, {x, 14, 74}, PlotStyle -> Directive[Gray, Dashed]],
   FrameLabel -> {"WorkingPrecision (digits)", "correct digits of T"}, PlotRange -> {{14, 74}, {0, 80}},
   Epilog -> Text[Style["dashed: digits = WorkingPrecision", 10, Gray, FontFamily -> "Times New Roman"], Scaled[{0.62, 0.12}]],
   Evaluate[frameOpts]];
tolData = Table[{pg, Module[{s = NOLSimulate[NOLModel @@ #[[1]], #[[2]], #[[3]], 200, PrecisionGoal -> pg, AccuracyGoal -> pg + 2]},
      NOLEnergyDrift[NOLModel @@ #[[1]], s, "Samples" -> 4001]]} & /@ {{{1, 0.4, 0.2}, 0.8, 0.1}, {{-1, 1, 0}, 1.6, 0.}, {{1, -4, 3}, 1.2, 0.3}},
   {pg, 6, 14, 1}];
f7b = ListLogPlot[Transpose[tolData], Joined -> True,
   PlotMarkers -> Automatic, PlotStyle -> cols[[{1, 4, 5}]],
   FrameLabel -> {"PrecisionGoal", "relative energy drift on [0, 200]"},
   PlotLegends -> leg[cols[[{1, 4, 5}]], {"a=1, b=0.4, c=0.2", "double well, outer", "triple well, outer"}, Scaled[{0.7, 0.8}]],
   Evaluate[frameOpts]];
save["fig7_precision_and_tolerance", row[lbl[f7a, "(a) arbitrary-precision period, cubic\[Dash]quintic"], lbl[f7b, "(b) RK8 energy drift vs tolerance"]]];

(* ---------------- Table: cross-implementation benchmark ---------------- *)
bench = Map[num, Rest@Import[FileNameJoin[{$root, "benchmarks", "python_periods.csv"}], "CSV"], {2}];
exactBench = <|"harmonic" -> {4, 0, 0, 2/5, 0}, "hardening" -> {1, 1, 0, 1, 0}, "softening" -> {1, -1/2, 0, 9/10, 0},
   "dw_inner" -> {-1, 1, 0, 6/5, 0}, "dw_outer" -> {-1, 1, 0, 8/5, 0}, "cq_demo" -> {1, 2/5, 1/5, 4/5, 1/10},
   "tw_center" -> {1, -4, 3, 3/10, 0}, "tw_side" -> {1, -4, 3, 11/10, 0}, "tw_outer" -> {1, -4, 3, 6/5, 3/10}|>;
fmtE[x_] := If[x == 0, "0", Module[{e = Floor[Log10[N[x]]]}, ToString[NumberForm[N[x/10^e], {3, 1}]] <> "e" <> ToString[e]]];
texE[x_] := If[x == 0, "0", Module[{e = Floor[Log10[N[x]]]}, "$" <> ToString[NumberForm[N[x/10^e], {3, 1}]] <> "\\times10^{" <> ToString[e] <> "}$"];
benchRows = Table[Module[{id = r[[1]], e, t40, tw1, cf, cfErr},
    e = exactBench[id];
    t40 = NOLPeriod[NOLModel @@ e[[1 ;; 3]], e[[4]], e[[5]], WorkingPrecision -> 40];
    tw1 = NOLPeriod[NOLModel @@ N[e[[1 ;; 3]]], N[e[[4]]], N[e[[5]]]];
    cf = Quiet[NOLExactSolution @@ N[e]];
    cfErr = If[FailureQ[cf], Missing[], Quiet@NOLExactSolutionError[NOLModel @@ N[e[[1 ;; 3]]], N[e[[4]]], N[e[[5]]], cf, 20]];
    <|"case" -> id, "a" -> e[[1]], "b" -> e[[2]], "c" -> e[[3]], "x0" -> e[[4]], "v0" -> e[[5]],
      "T40" -> ToString[NumberForm[N[t40, 20], 16]],
```

```
      "errPy" -> N[Abs[r[[7]] - t40]/t40], "errWL" -> N[Abs[twl - t40]/t40], "errCF" -> cfErr|>], {r, bench}];
Export[FileNameJoin[{$tab, "benchmark_periods.csv"}],
  Prepend[Table[{b["case"], ToString[b["a"], InputForm], ToString[b["b"], InputForm], ToString[b["c"], InputForm],
     ToString[b["x0"], InputForm], ToString[b["v0"], InputForm], b["T40"], fmtE[b["errPy"]], fmtE[b["errWL"]],
     If[MissingQ[b["errCF"]], "n/a", fmtE[b["errCF"]]]}, {b, benchRows}],
   {"case", "a", "b", "c", "x0", "v0", "T_40digit", "relerr_python", "relerr_wolfram", "closed_form_maxerr_0_20"}], "CSV"];
Export[FileNameJoin[{$tab, "benchmark_periods.tex"}],
  StringRiffle[Table[StringRiffle[{StringReplace[b["case"], "_" -> "\\_"],
      "$" <> ToString[TeXForm[b["a"]]] <> "$", "$" <> ToString[TeXForm[b["b"]]] <> "$", "$" <> ToString[TeXForm[b["c"]]] <> "$",
      "$" <> ToString[TeXForm[b["x0"]]] <> "$", "$" <> ToString[TeXForm[b["v0"]]] <> "$", b["T40"],
      texE[b["errPy"]], texE[b["errWL"]], If[MissingQ[b["errCF"]], "n/a", texE[b["errCF"]]]}, " & "] <> " \\\\",
     {b, benchRows}], "\n"], "Text"];
log["tables written"];
Export[FileNameJoin[{$tab, "environment.txt"}],
  StringJoin["Generated by MakeFigures.wls with ", $Version, "\nPackage version ", NOLVersion, "\nDate ", DateString[], "\n"], "Text"];
(* ---------------- every number quoted in the manuscript ---------------- *)
texM[x_] := If[x == 0, "0", Module[{e = Floor[Log10[N[x]]]},
    "\\ensuremath{" <> ToString[NumberForm[N[x/10^e], {2, 1}]] <> "\\times10^{" <> ToString[e] <> "}}"]];
errLegacy = Flatten@Table[MapThread[relE, {legacyByF[f], refT[f]}], {f, families}];
errPy = Flatten@Table[MapThread[relE, {famData[f][[All, 7]], refT[f]}], {f, families}];
errWL = Flatten@Table[MapThread[relE, {wlT[f], refT[f]}], {f, families}];
Export[FileNameJoin[{$tab, "period_accuracy.csv"}],
  Prepend[Flatten[Table[Transpose[{ConstantArray[f, Length[famData[f]]], famData[f][[All, 6]], ToString[N[#, 20], InputForm] & /@ refT[f],
      MapThread[relE, {legacyByF[f], refT[f]}], MapThread[relE, {famData[f][[All, 7]], refT[f]}], MapThread[relE, {wlT[f], refT[f]}]}], {f, families}], 1],
   {"family", "E", "T_reference", "relerr_python_v010_sin_substitution", "relerr_python_factored", "relerr_wolfram_factored"}], "CSV"];
testRows = Rest@Import[FileNameJoin[{$dir, "test_report.csv"}], "CSV"];
repParts = StringSplit[exDemo["Representation"], {" / y: ", ":"}];
numbers = {
   "wlTests" -> ToString[Length[testRows]], "wlPassed" -> ToString[Count[testRows[[All, 2]], "Success"]],
   "pythonTests" -> "15",
   "pythonLegacyMaxErr" -> texM[Max[errLegacy]], "pythonLegacyMedErr" -> texM[Median[errLegacy]],
   "pythonMaxErr" -> texM[Max[errPy]], "pythonMedErr" -> texM[Median[errPy]],
   "wlMaxErr" -> texM[Max[errWL]], "wlMedErr" -> texM[Median[errWL]],
   "campN" -> ToString[nCampaign], "campOK" -> ToString[campaignSummary["represented"]],
   "campMed" -> texM[campaignSummary["median_error"]], "campPninetyfive" -> texM[campaignSummary["p95_error"]],
   "campMax" -> texM[campaignSummary["max_error"]],
   "campBelow" -> ToString[Round[100 campaignSummary["fraction_below_1e-8"]]] <> "\\%",
   "campTime" -> ToString[NumberForm[1000 campaignSummary["median_time_s"], {3, 0}]] <> "~ms",
   "demoRep" -> "family~" <> repParts[[1]] <> ", \\texttt{" <> StringTrim[repParts[[2]]] <> "} branch",
   "mmaVersion" -> StringTrim[First@StringSplit[$Version, "("]]};
Export[FileNameJoin[{$tab, "numbers.tex"}],
  StringRiffle[("\\newcommand{\\" <> #[[1]] <> "}{" <> #[[2]] <> "}") & /@ numbers, "\n"] <> "\n", "Text"];
log["numbers ", numbers];
log["ALL DONE"];
```

```
#!/usr/bin/env wolframscript
(* Runs the test suite and regenerates every figure and table.
   Usage (from any folder):  wolframscript -file RunAll.wls *)
$dir = DirectoryName[$InputFileName];
Print["== NonlinearOscillatorLab: tests =="];
Get[FileNameJoin[{$dir, "RunTests.wls"}]];
Print["== NonlinearOscillatorLab: figures and tables =="];
Get[FileNameJoin[{$dir, "MakeFigures.wls"}]];
Print["== finished =="];
```